

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Домашнее задание 1 по дисциплине Теория функции комплексного переменного

Выполнили: Снагин Станислав
Максимович Максимович
Поток: 21.2

Санкт Петербург 2025 г.

Содержание

Ход работы.....	3
Задание 1. Линейные фильтры.....	3
Фильтр первого порядка.....	3
Режекторный полосовой фильтр.....	15
Задание 2. Сглаживание биржевых данных.....	21
Приложения.....	23

Ход работы

Задание 1. Линейные фильтры

Задайтесь числами a , t_1 , t_2 такими, что $t_1 < t_2$, и рассмотрите функцию

$$g(t) = \begin{cases} a, & t \in [t_1, t_2], \\ 0, & t \notin [t_1, t_2]; \end{cases}$$

и ее зашумленную версию

$$u(t) = g(t) + b\xi(t) + c \sin(dt),$$

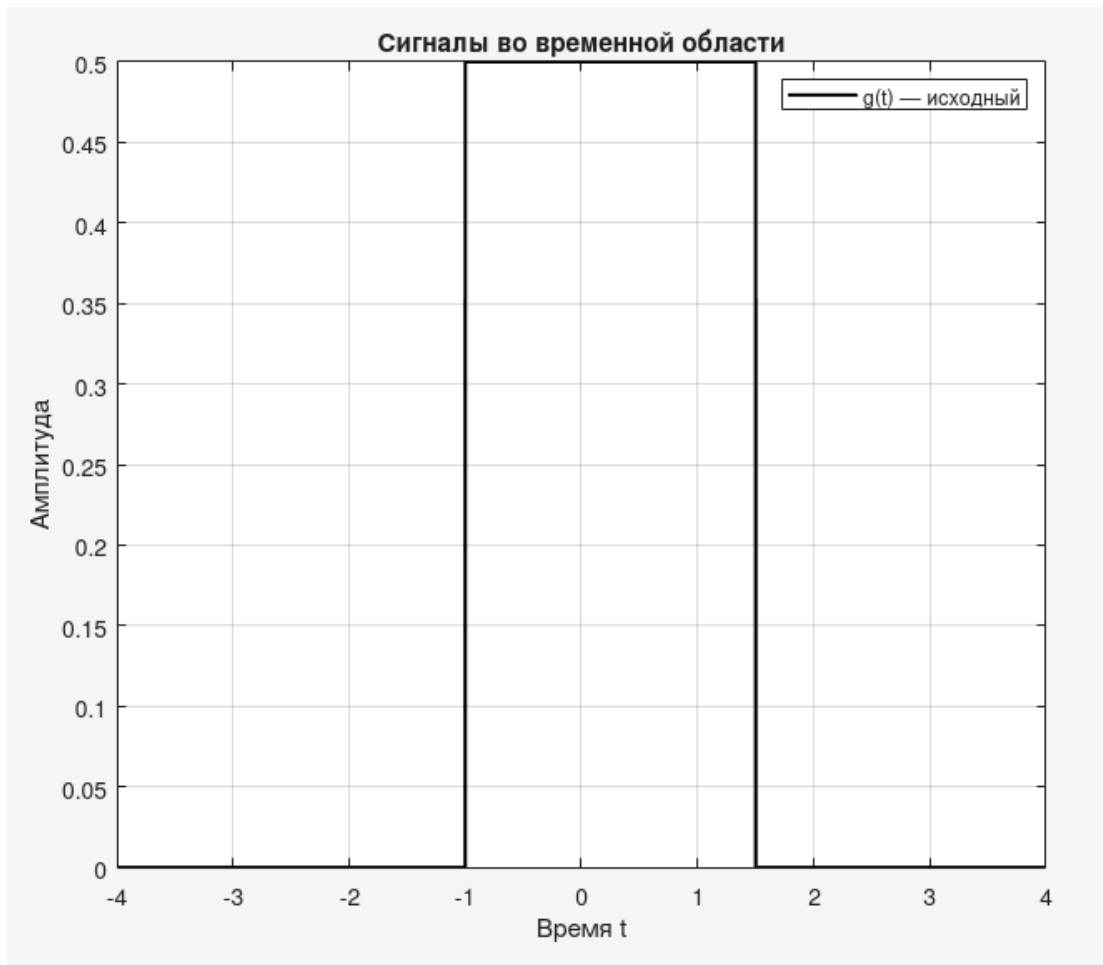
где $\xi(t) \sim \mathcal{U}[-1, 1]$ – равномерное распределение, представляющее белый шум, а значения b , c , d – параметры возмущения.

рис. 1.0.1 графики $g(t)$ и $u(t)$

Фильтр первого порядка

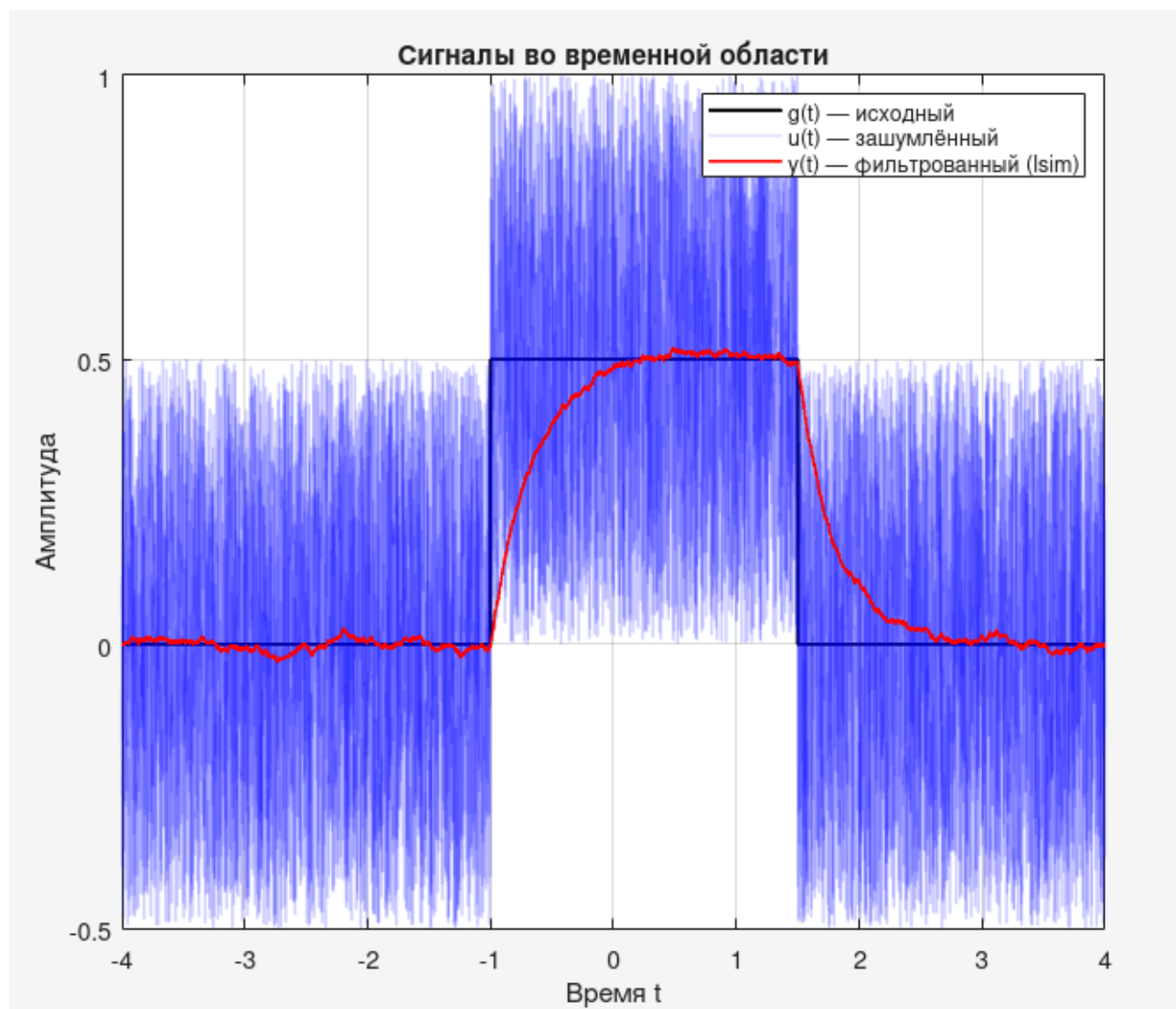
Код см. в приложении 1.1

Для примера зададимся числами $a = 0.5$, $t_1 = -1$, $t_2 = 1.5$; $b = 0.5$, $c = 0$, $d = 0$. График $g(t)$ будет выглядеть следующим образом:



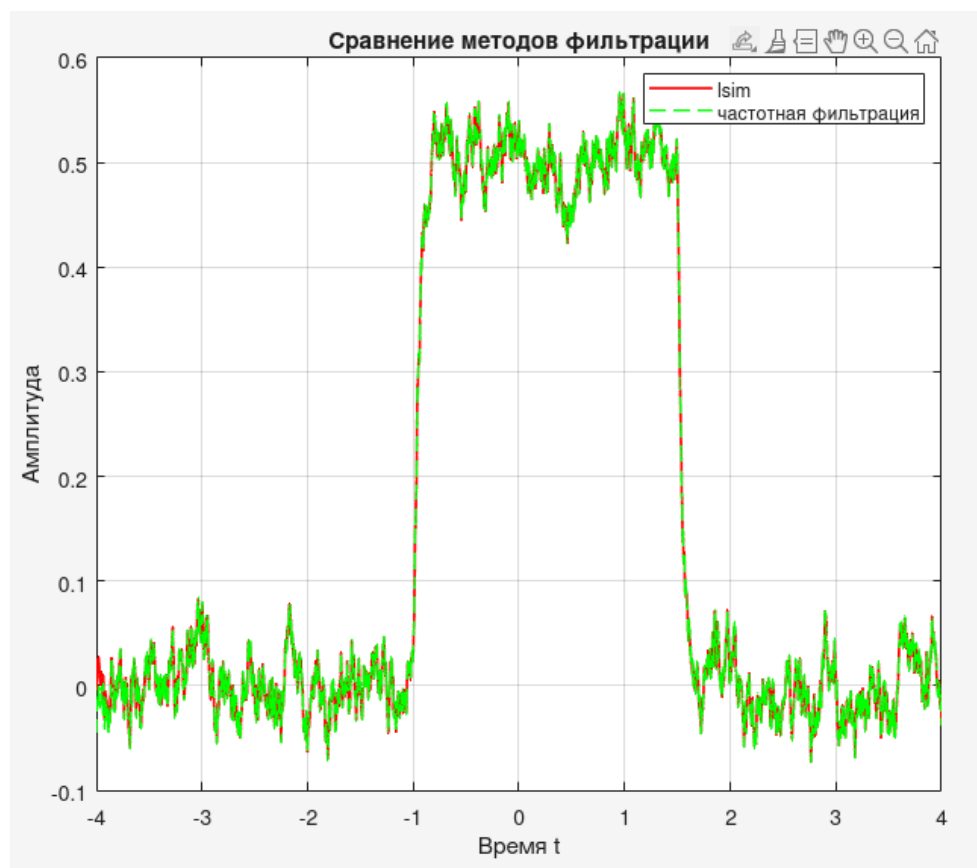
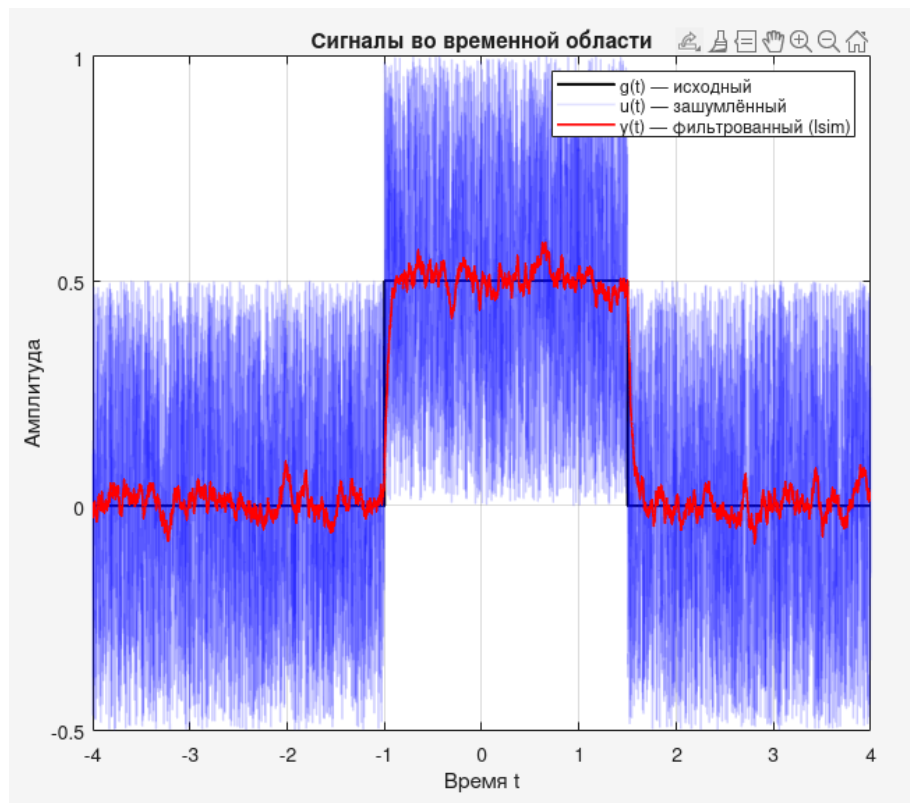
Зададим постоянную времени $T = 0.3$ и пропустим сигнал $u(t)$ через линейный фильтр первого порядка:

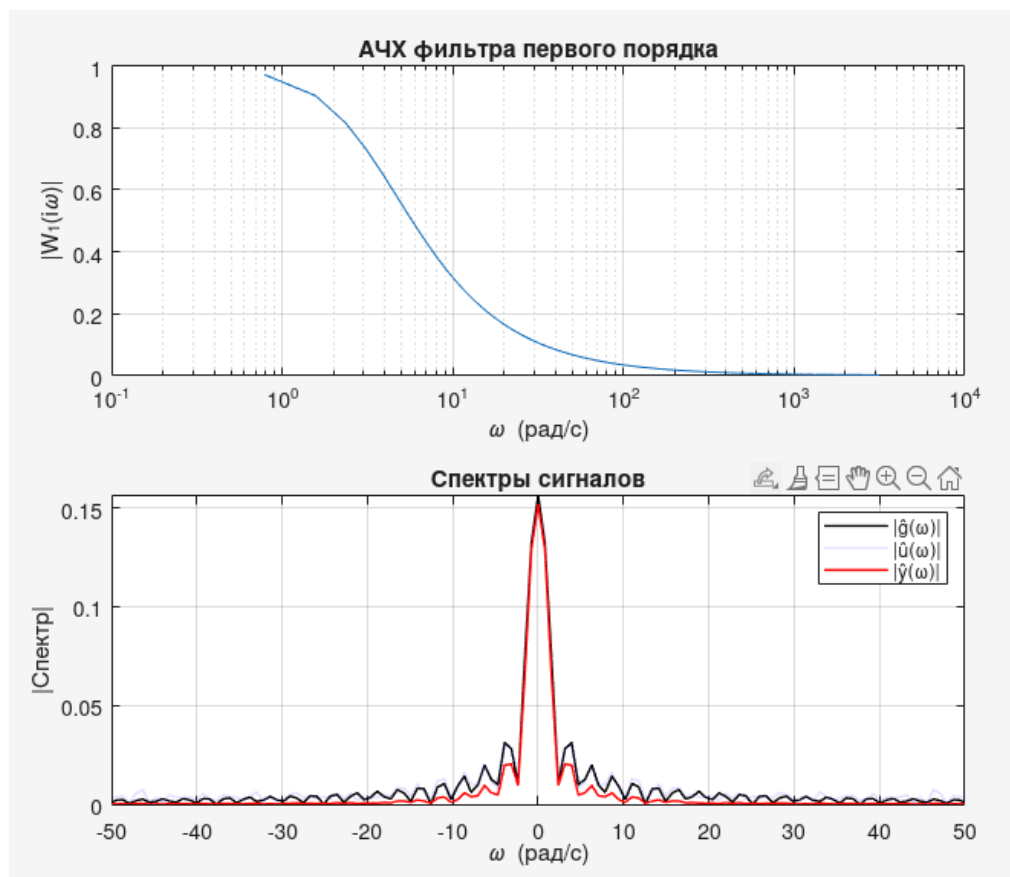
$$W_1(p) = \frac{1}{Tp + 1}.$$



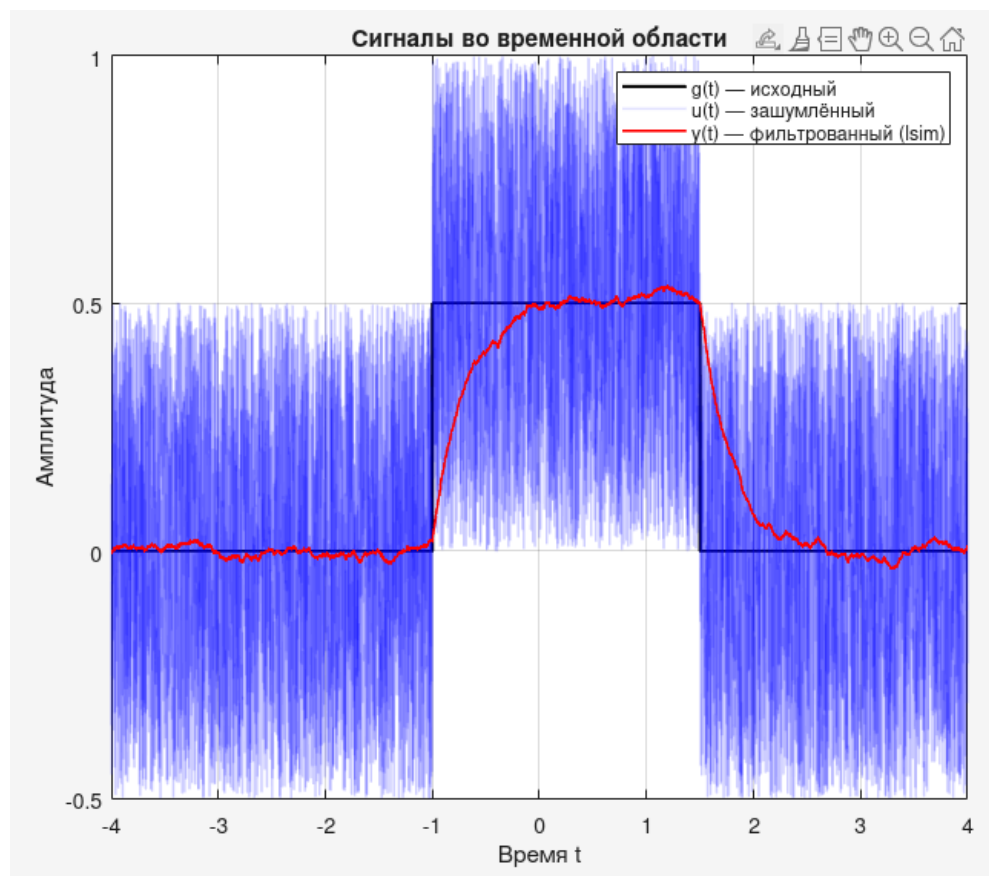
Проведем несколько экспериментов с разными значениями параметра T ($\alpha = 0.5$):

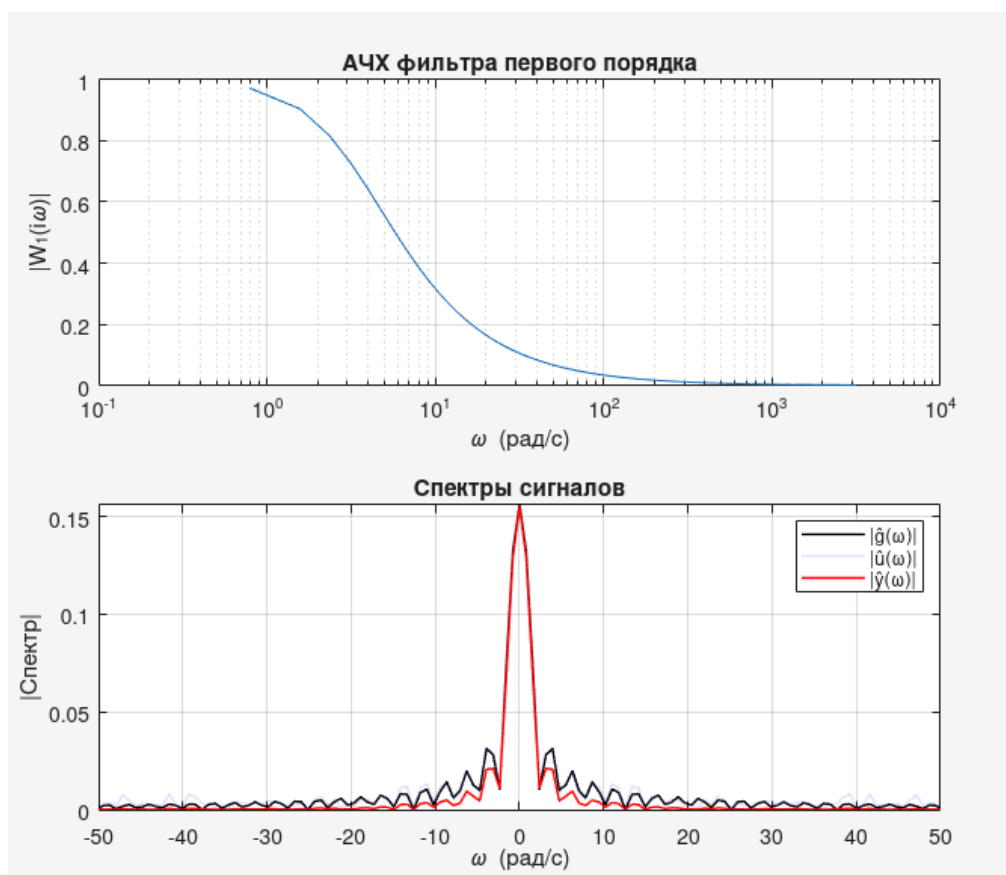
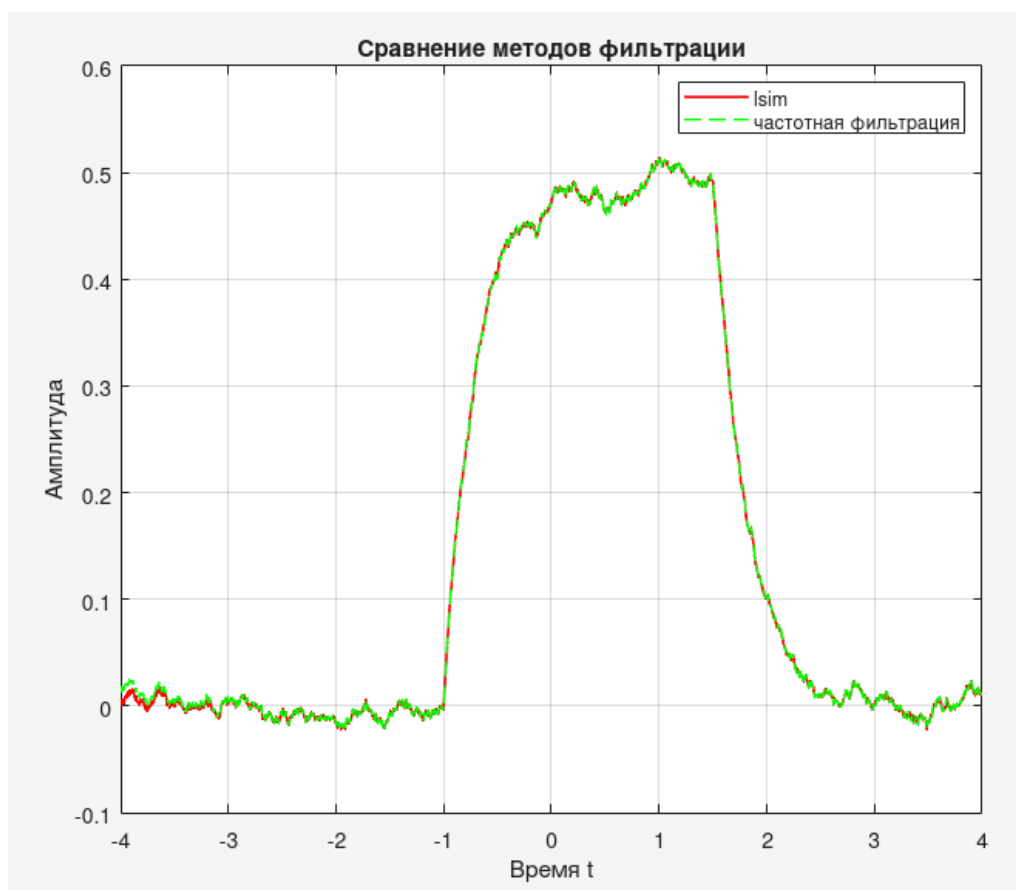
$T = 0.05$ Слабое сглаживание: шум почти не убирается, но форма сигнала почти не искажена



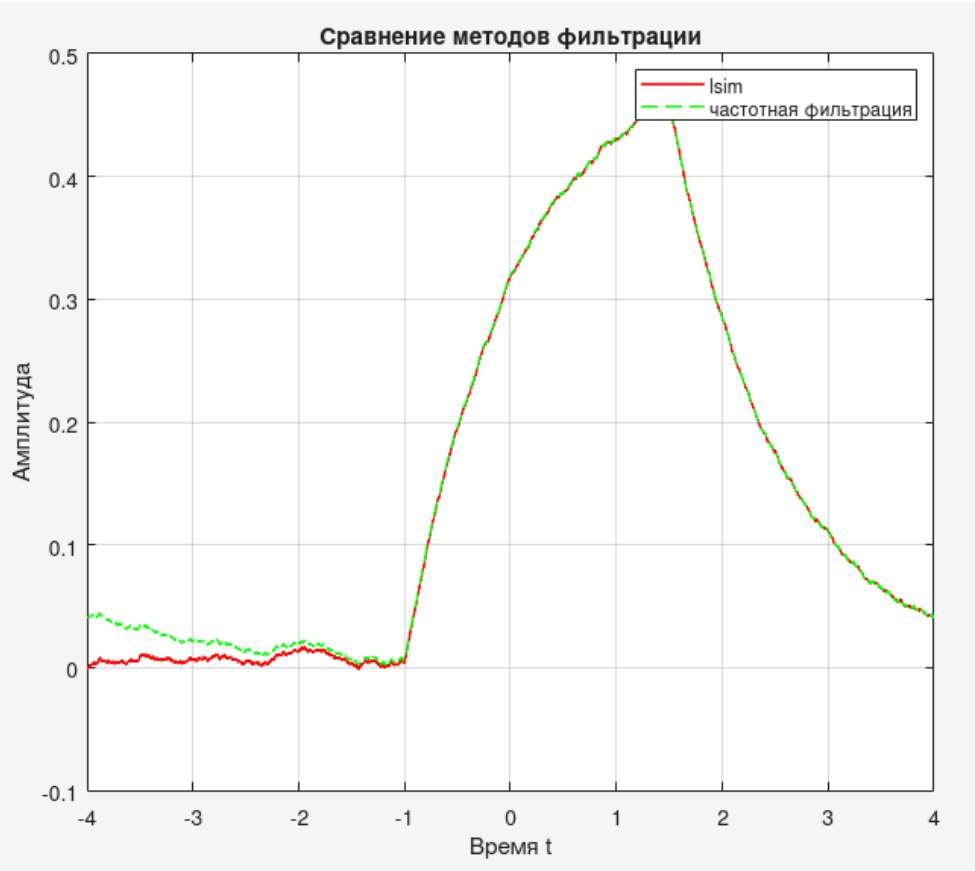
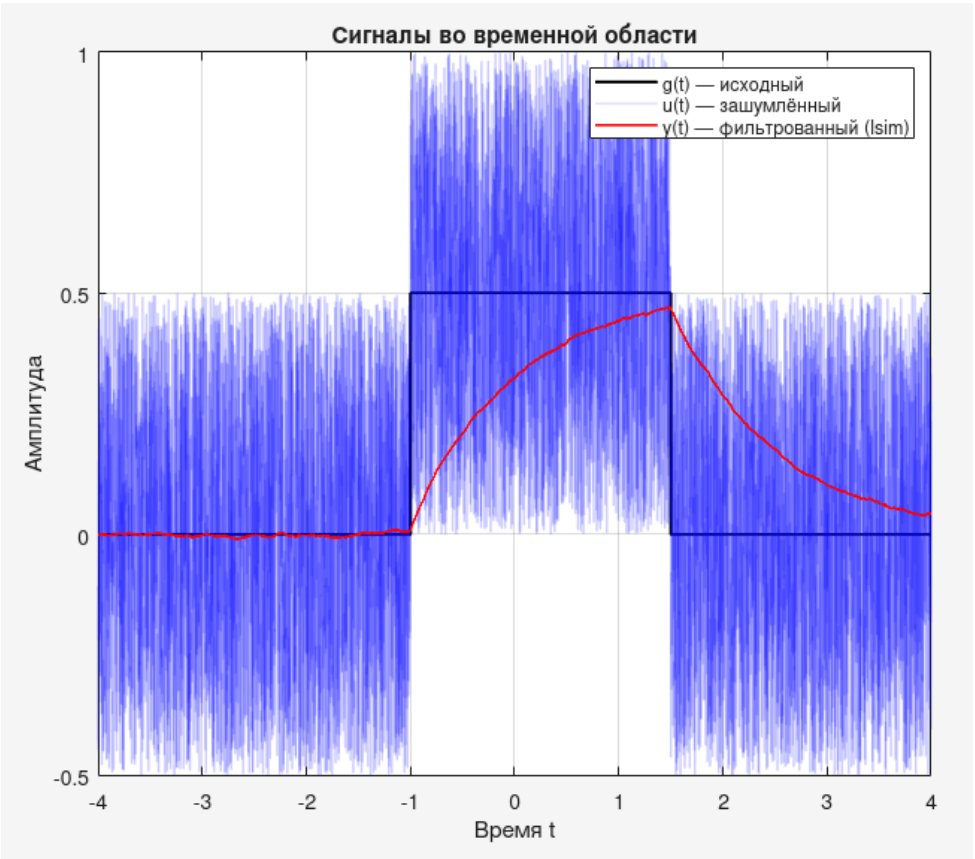


$T = 0.3$ Среднее сглаживание: шум уменьшен, фронты мягкие

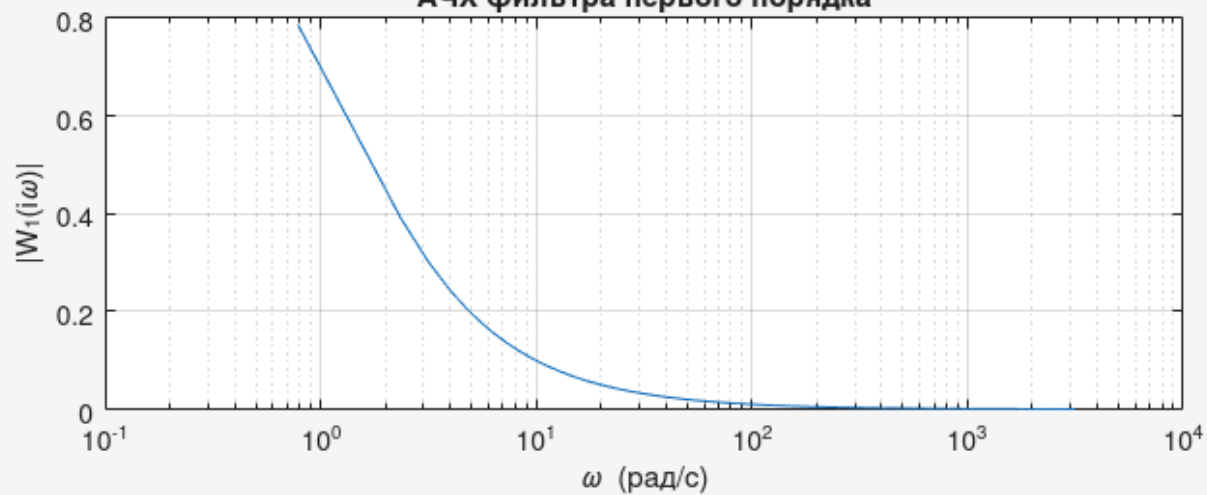




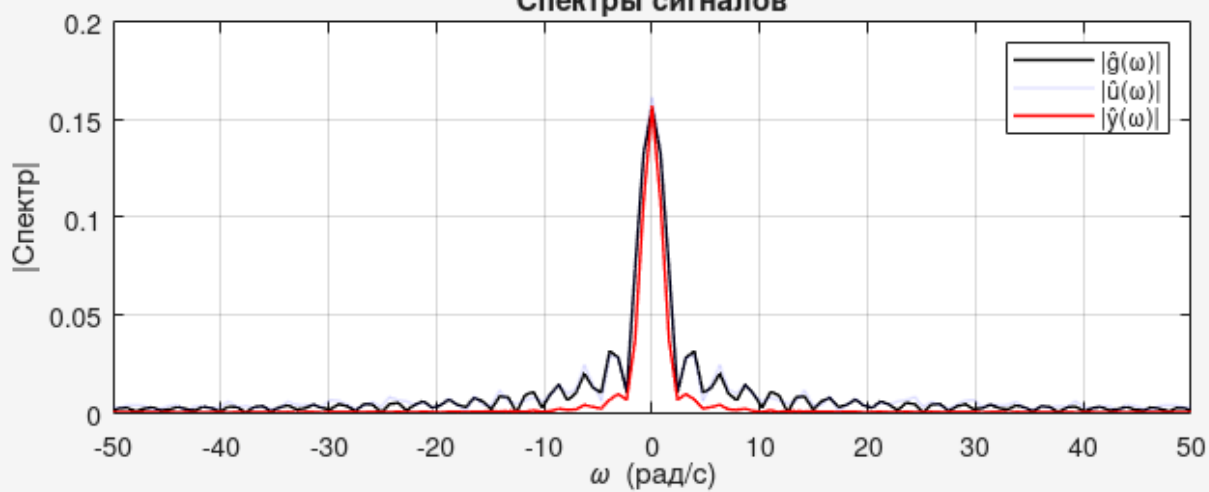
T = 1.0 Сильное сглаживание: шума почти нет, импульс размыт



АЧХ фильтра первого порядка

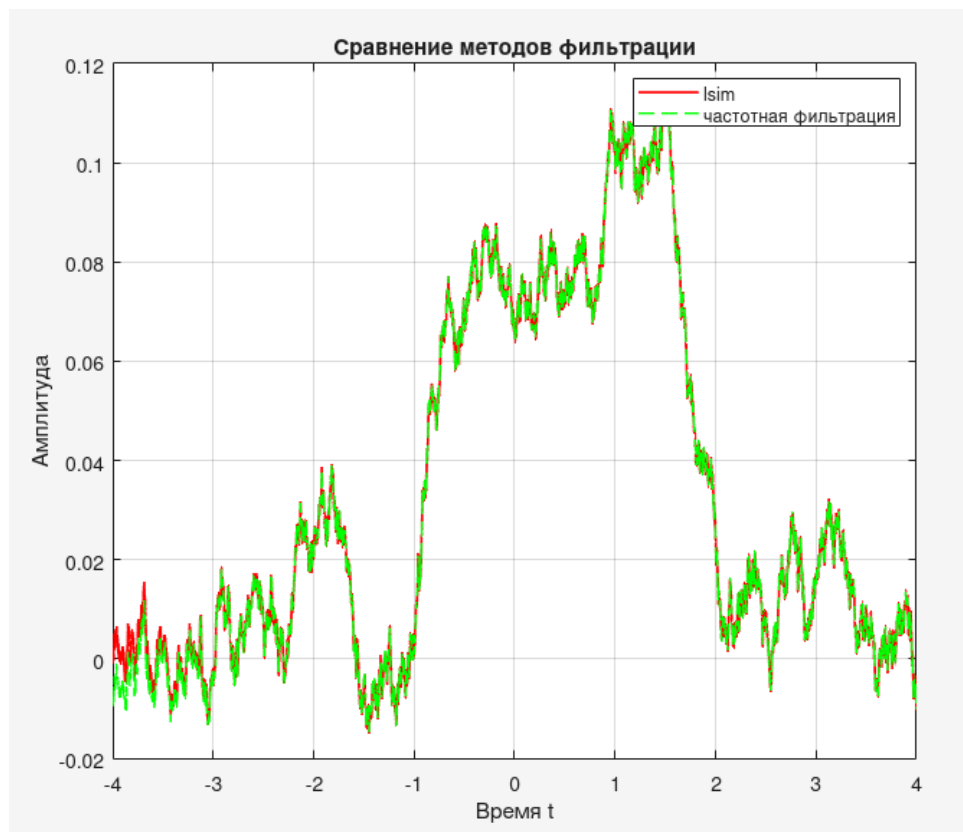
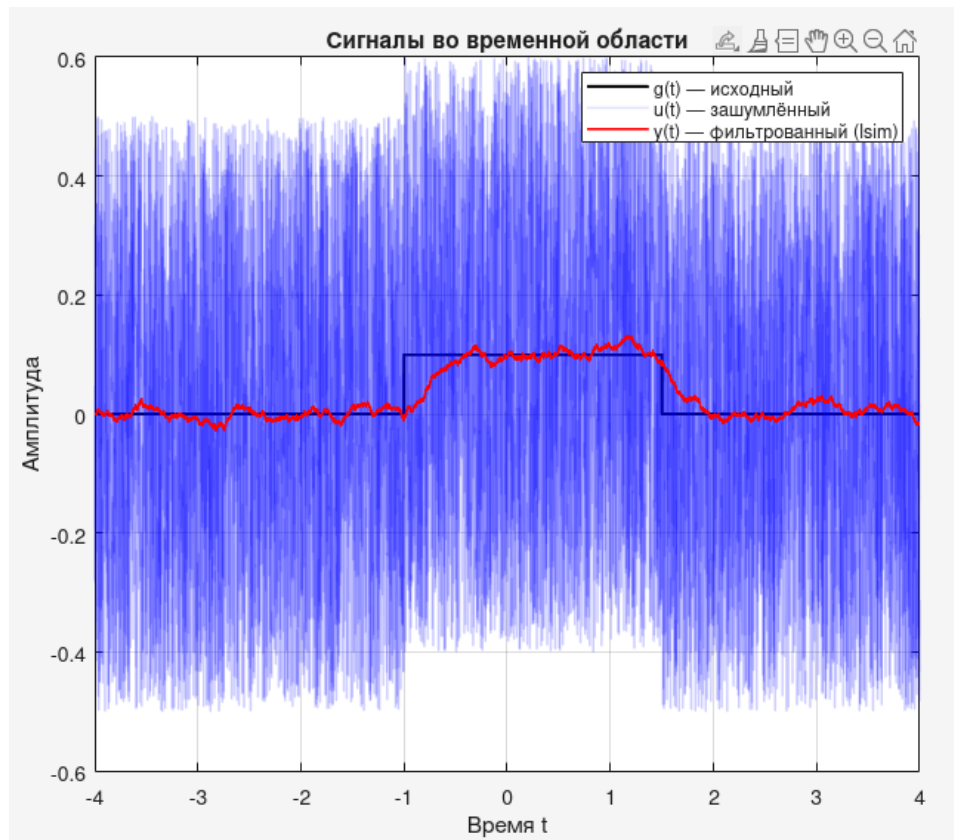


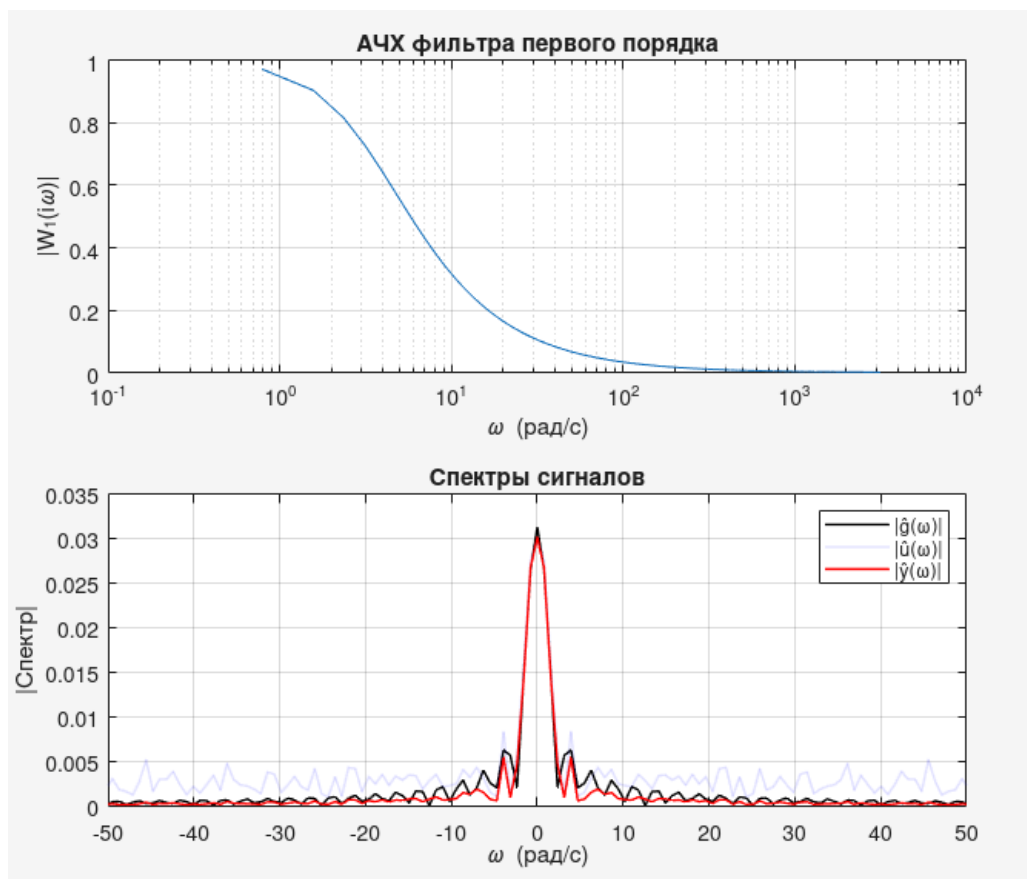
Спектры сигналов



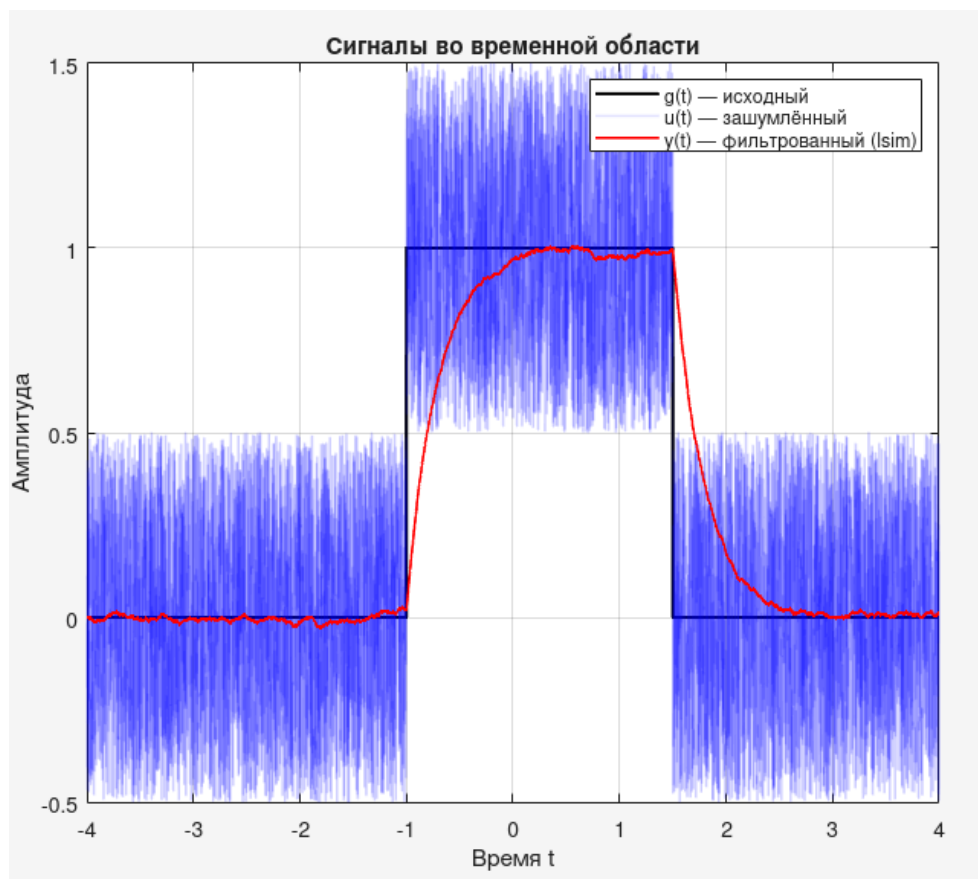
Теперь зафиксируем параметр T ($T = 0.3$) и будем изменять параметр a .

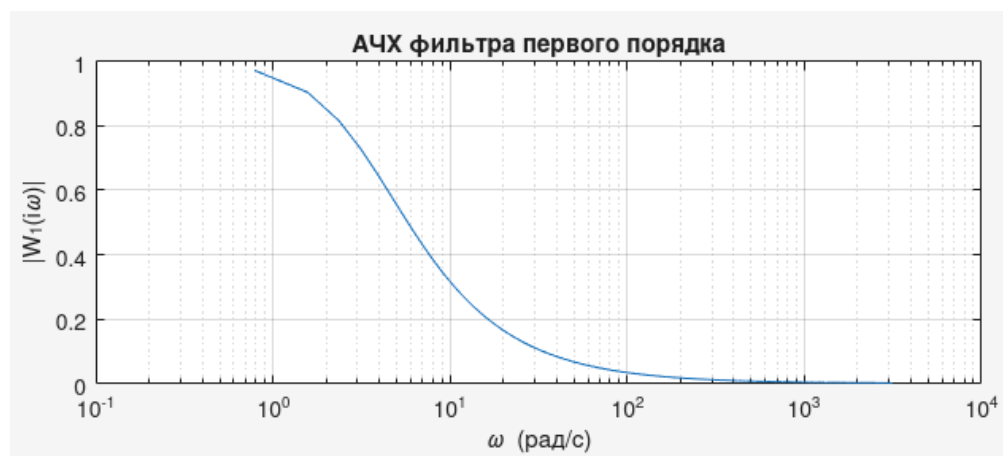
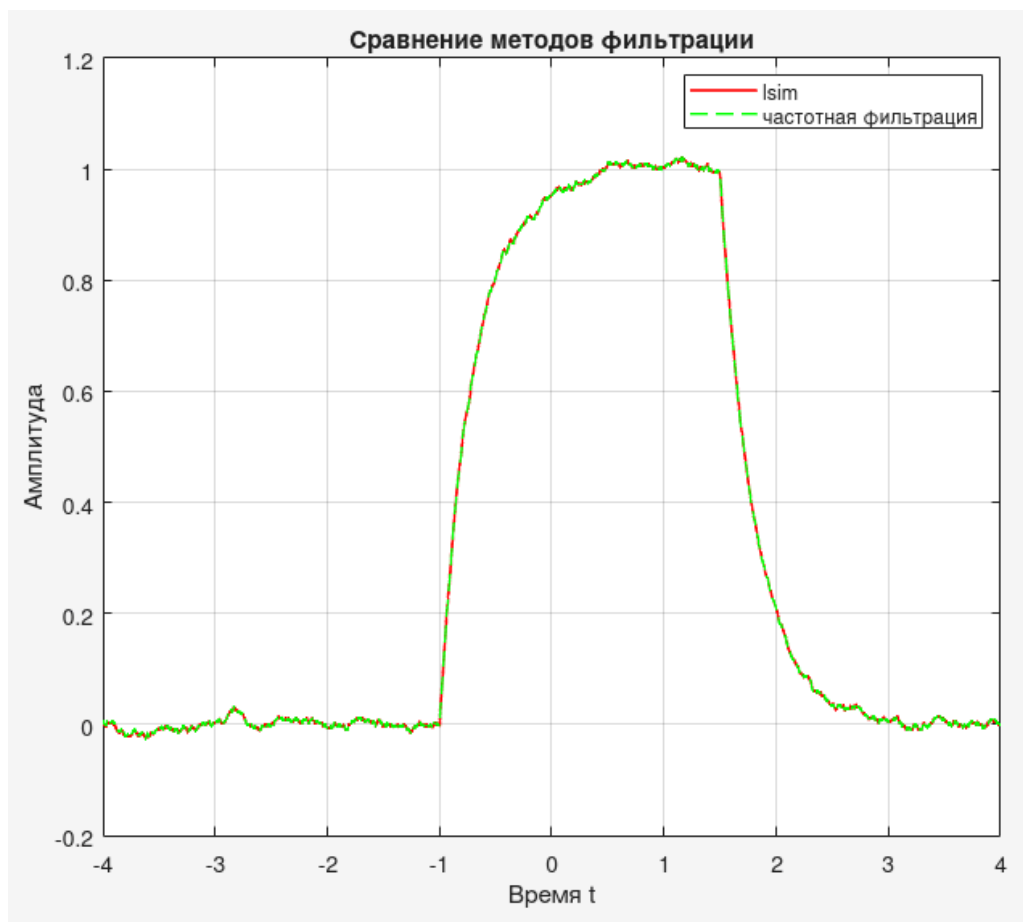
$a = 0.1$ Так как амплитуда импульса маленькая, шум может его заглушить. Даже после фильтрации функцию невозможно уверенно идентифицировать.



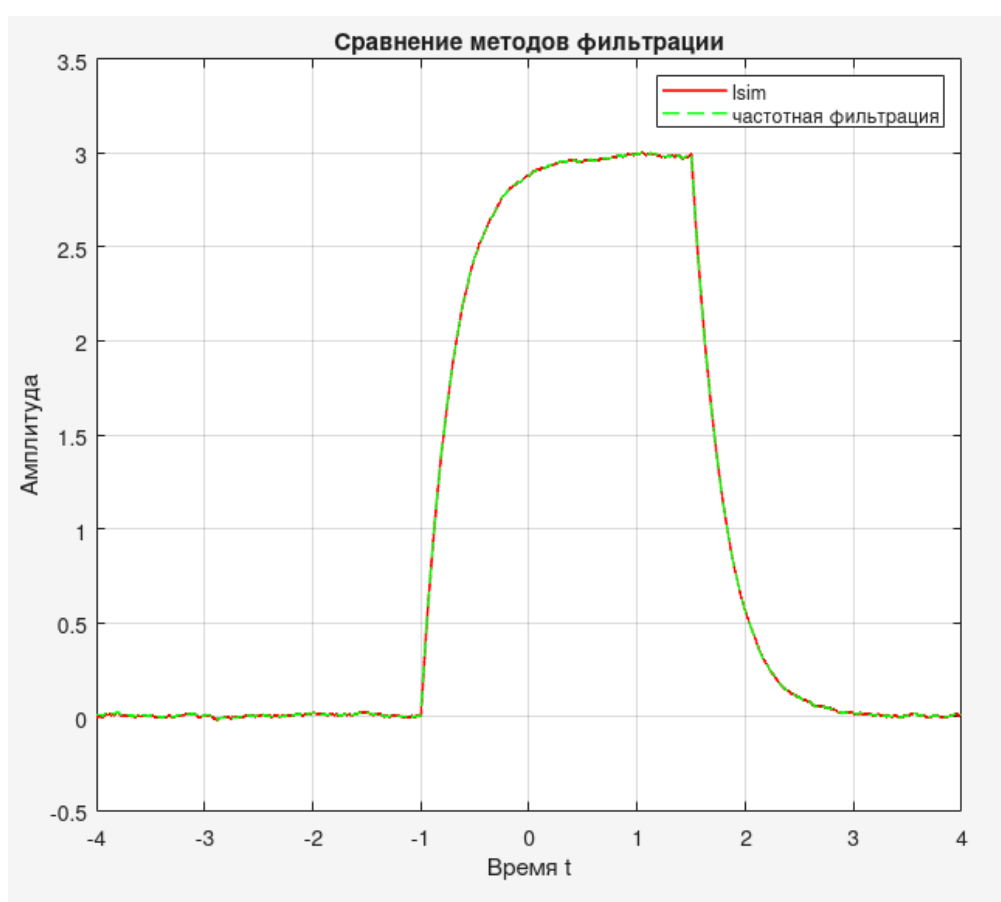
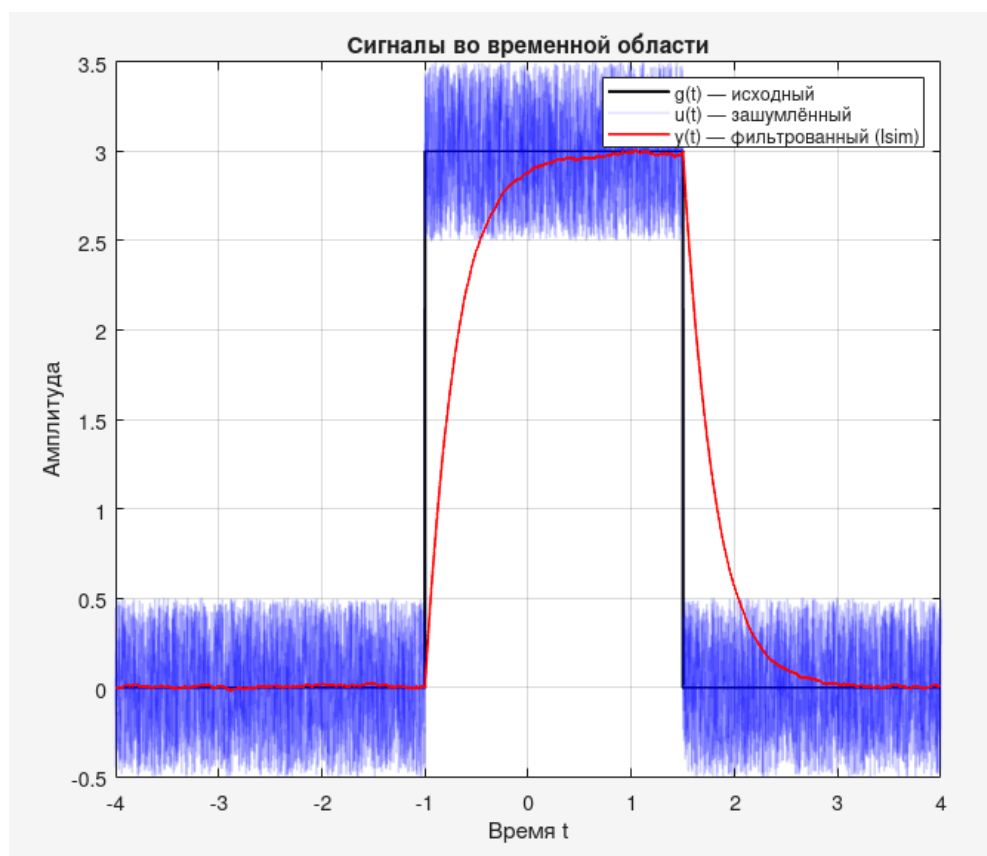


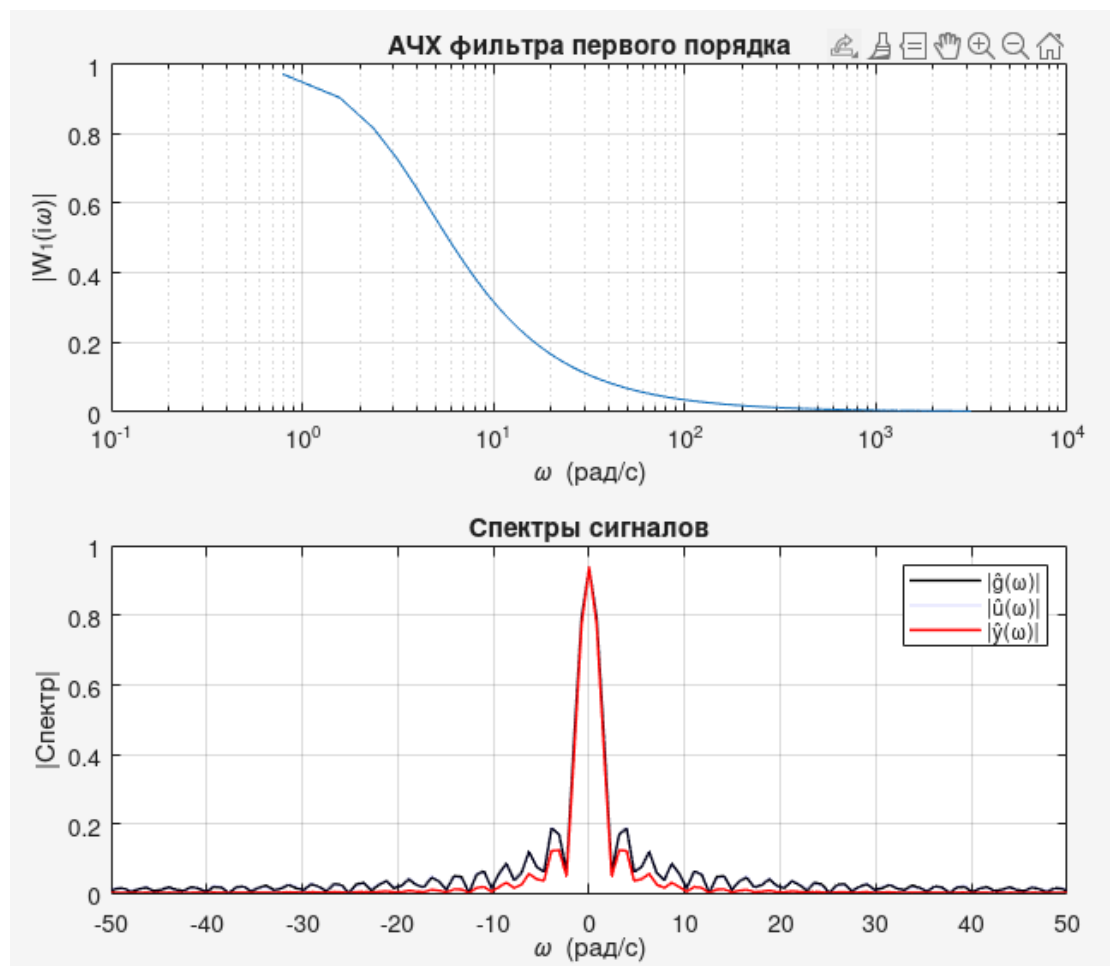
a = 1 Импульс различим, уже можно однозначно понять где функция.





a = 3 Импульс явно различим





Выводы по п. 1.1

Фильтр первого порядка эффективно подавляет шум, однако его действие зависит от постоянной времени T : малые значения T слабо сглаживают шум, но сохраняют форму сигнала, а большие - сильнее подавляют шум, но искажают импульс. Также установлено, что при малой амплитуде сигнала он теряется в шуме, при $a \geq 1$ фильтрация позволяет успешно восстановить его форму. Подтверждена эквивалентность временного и частотного методов фильтрации.

Режекторный полосовой фильтр

Код см. в приложении 1.2

Примем $b = 0$ и рассмотрим линейный фильтр вида

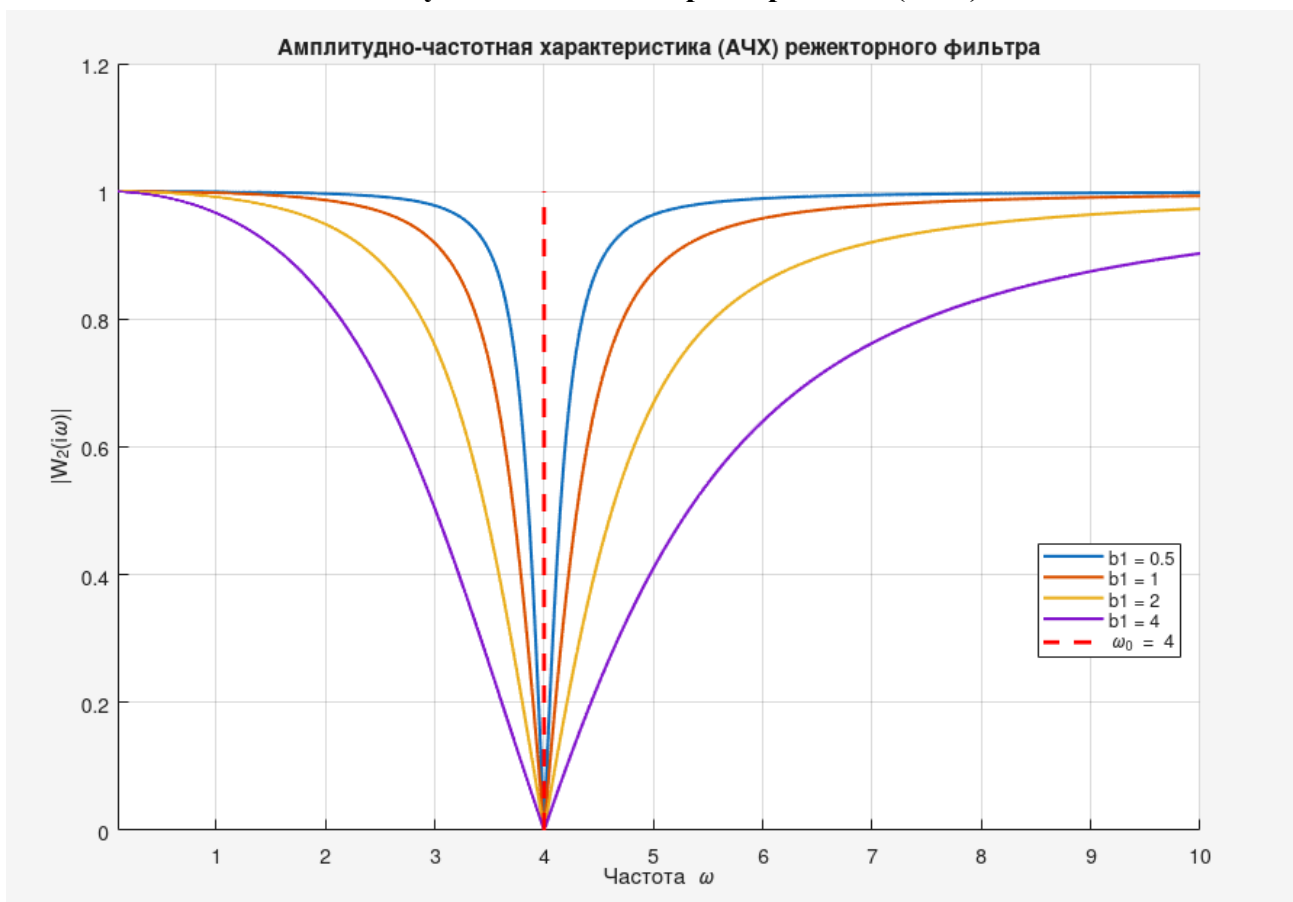
$$W_2(p) = \frac{p^2 + a_1p + a_2}{p^2 + b_1p + b_2}.$$

Выберем значения параметров $a_1, a_2, b_1, b_2 \in \mathbb{R}$ исходя из условий:

- Фильтр должен быть устойчивым (корни полинома знаменателя p_1, p_2 должны иметь строго отрицательную вещественную часть).
- Для нижних частот ($\omega \rightarrow 0$) и верхних частот ($\omega \rightarrow \infty$) АЧХ фильтра должна быть равна 1.
- Для некоторой частоты ω_0 АЧХ фильтра должна быть равна 0.

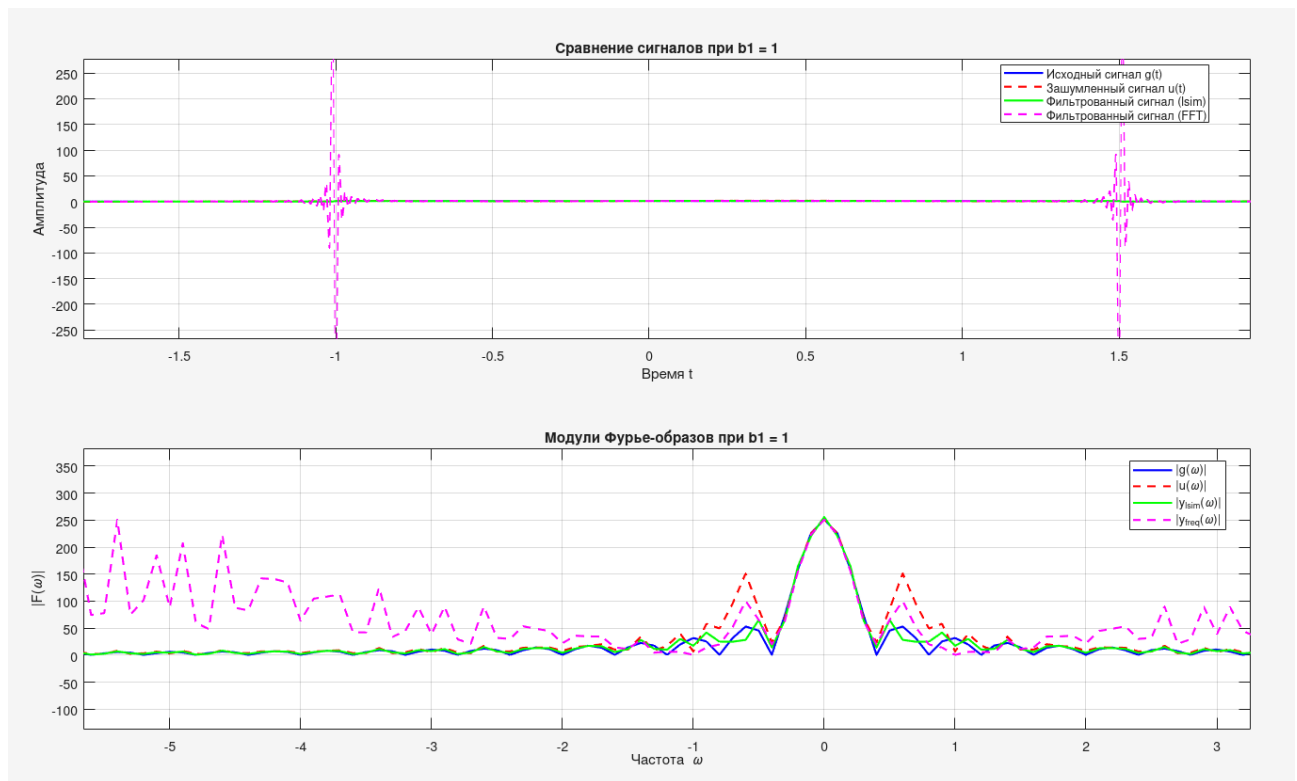
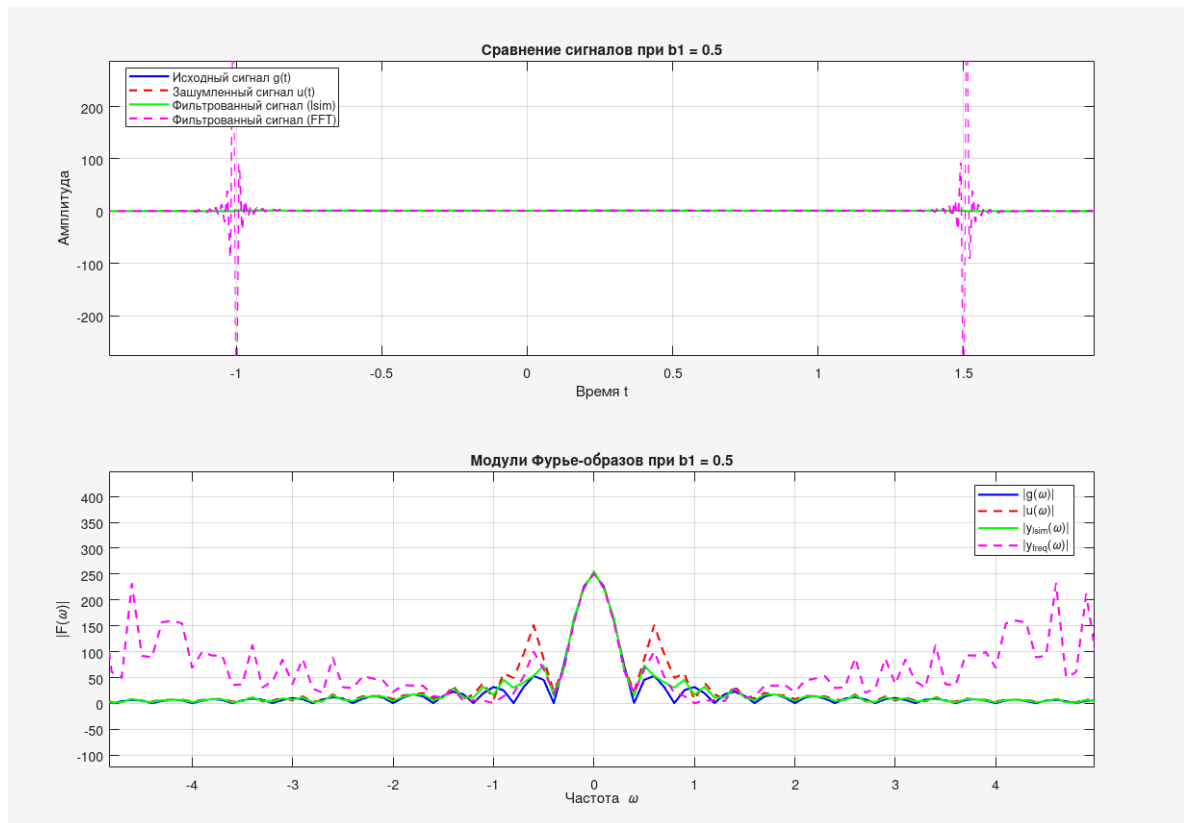
Попробуем использовать фильтр, который имеет нули в числите на нужной частоте ω_0 , а полюса в знаменателе — немного ниже этой частоты (для устойчивости). Т.е. возьмем $a_1 = 0, a_2 = \omega_0^2, b_1$ - переменное значение (параметр демпфирования, $b_1 > 0$).

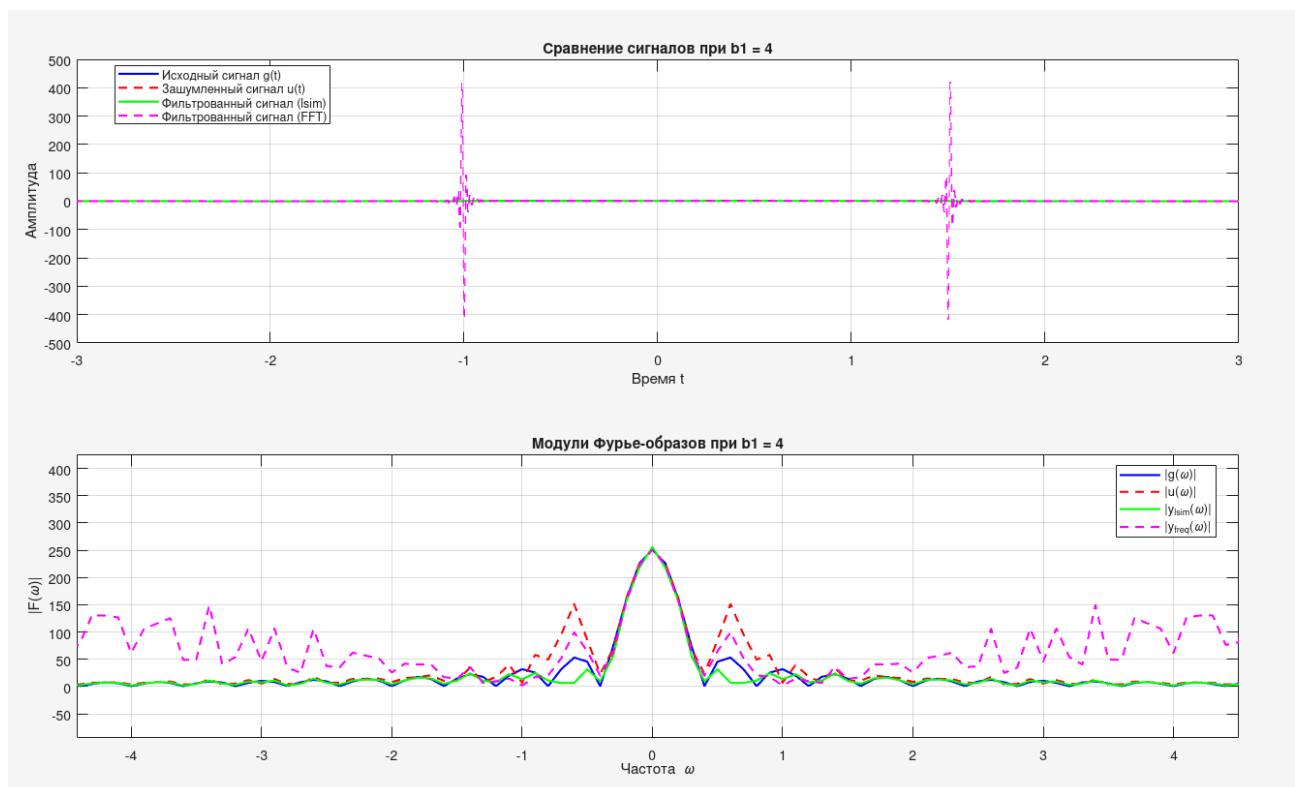
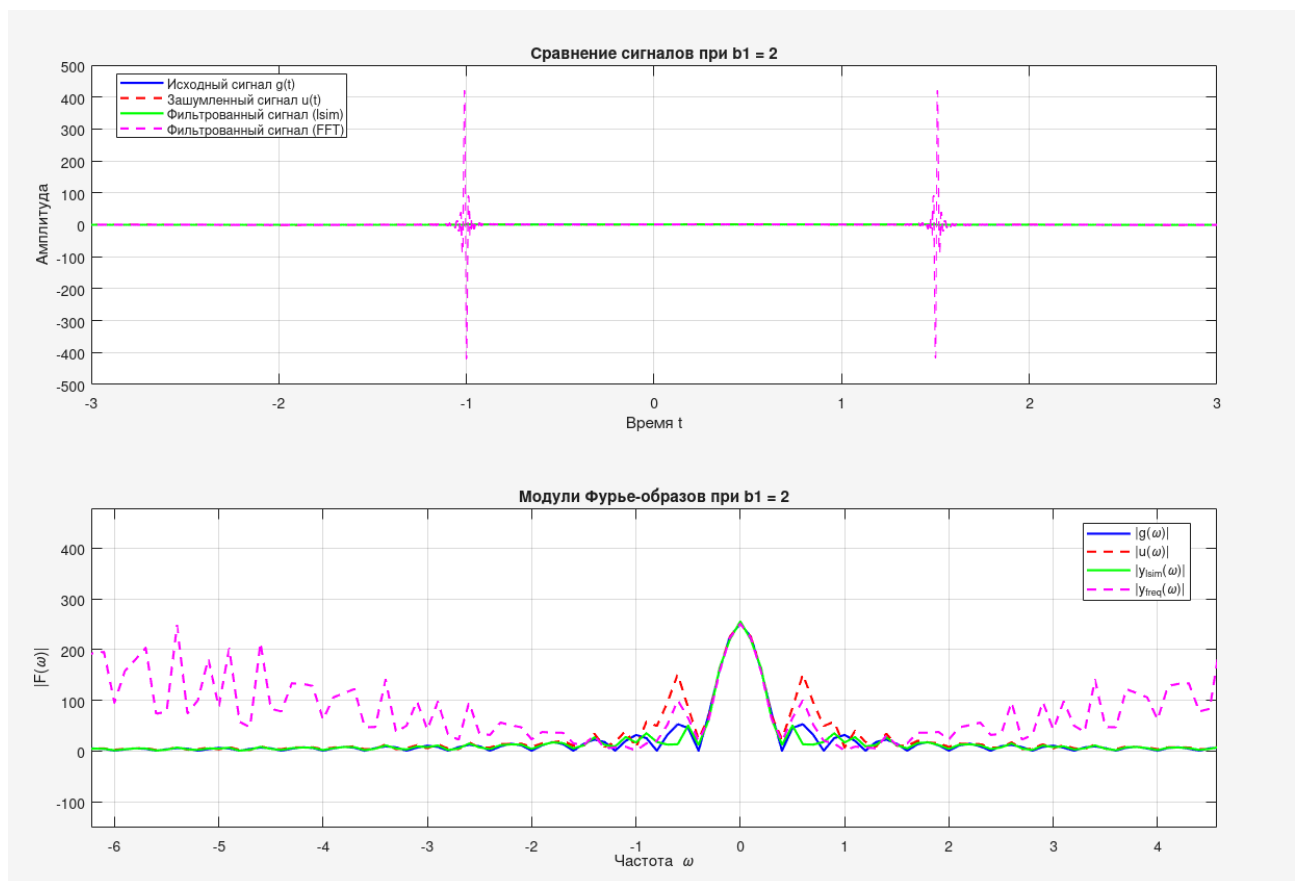
Амплитудно-частотная характеристика (АЧХ)



Можно заметить, что при малых b_1 (0.5) есть узкий и глубокий провал на частоте ω_0 , тогда как при больших b_1 (4.0) - подавление хуже, но меньше искажений. На низких и высоких частотах АЧХ стремится к 1, что обеспечивает пропускание основного сигнала.

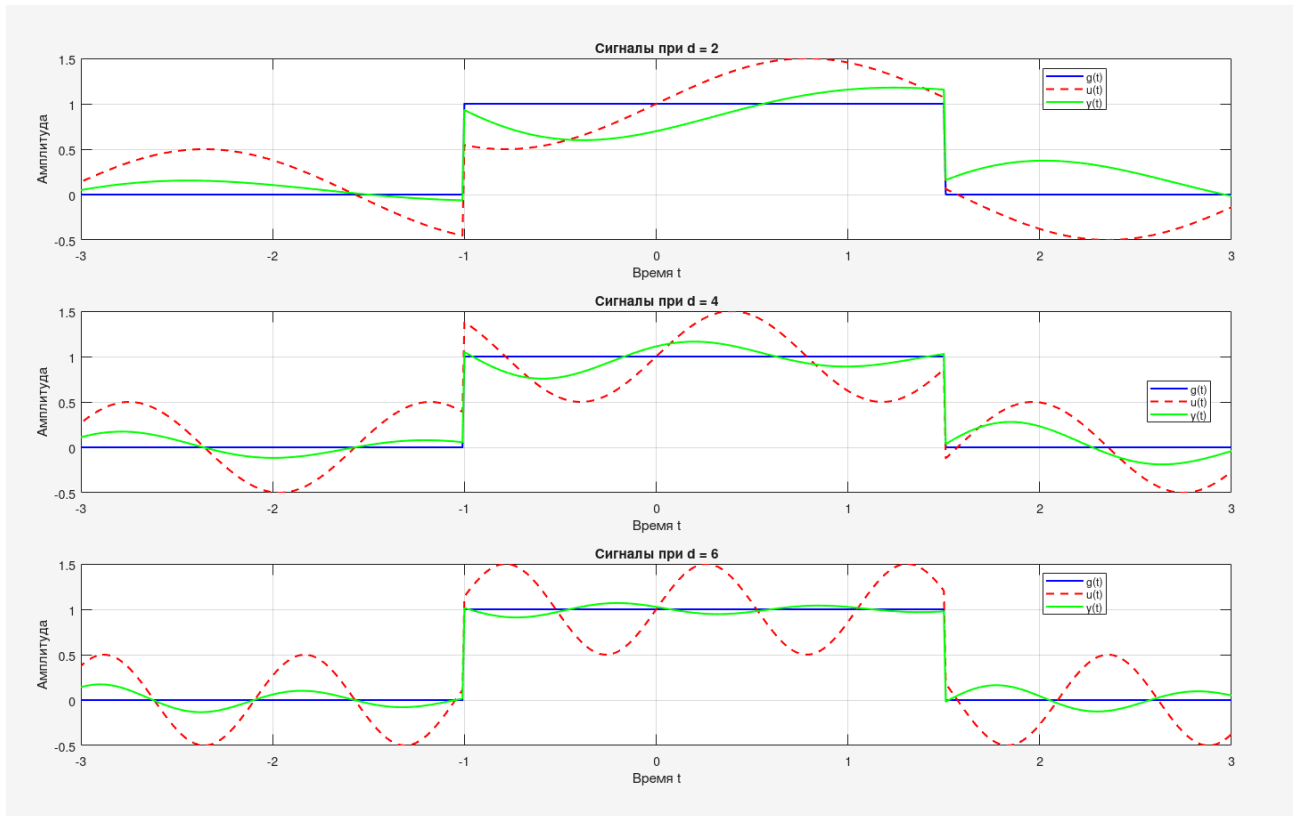
Сравнение сигналов во временной области





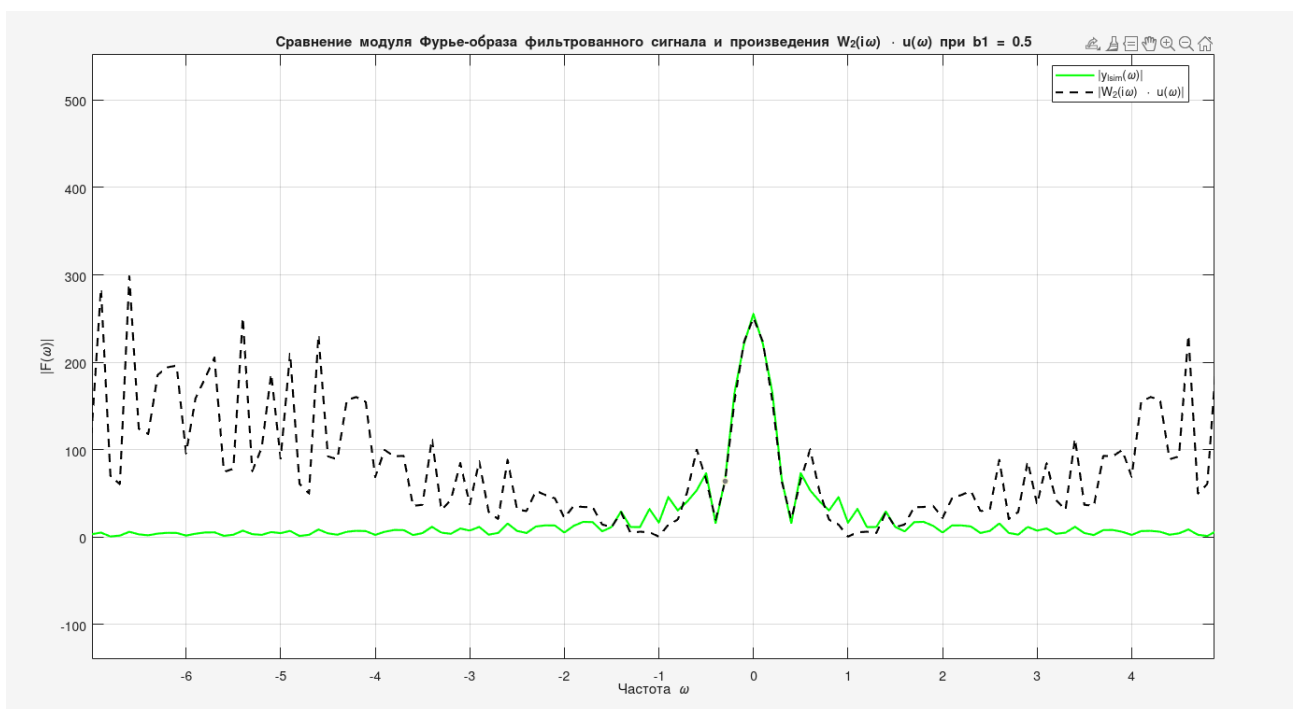
При малых значениях b_1 фильтр очень узкополосный. Он эффективно подавляет помеху на частоте ω_0 , но может искажать другие частоты сигнала, если они близки к ω_0 . При больших значениях b_1 фильтр широкополосный. Он слабее подавляет помеху, но меньше искажает основной сигнал. Оптимальное значение зависит от конкретной задачи в зависимости от того, что нужно: подавление помехи или сохранение формы сигнала (или их баланс).

Сигналы ($d = \text{const}$)

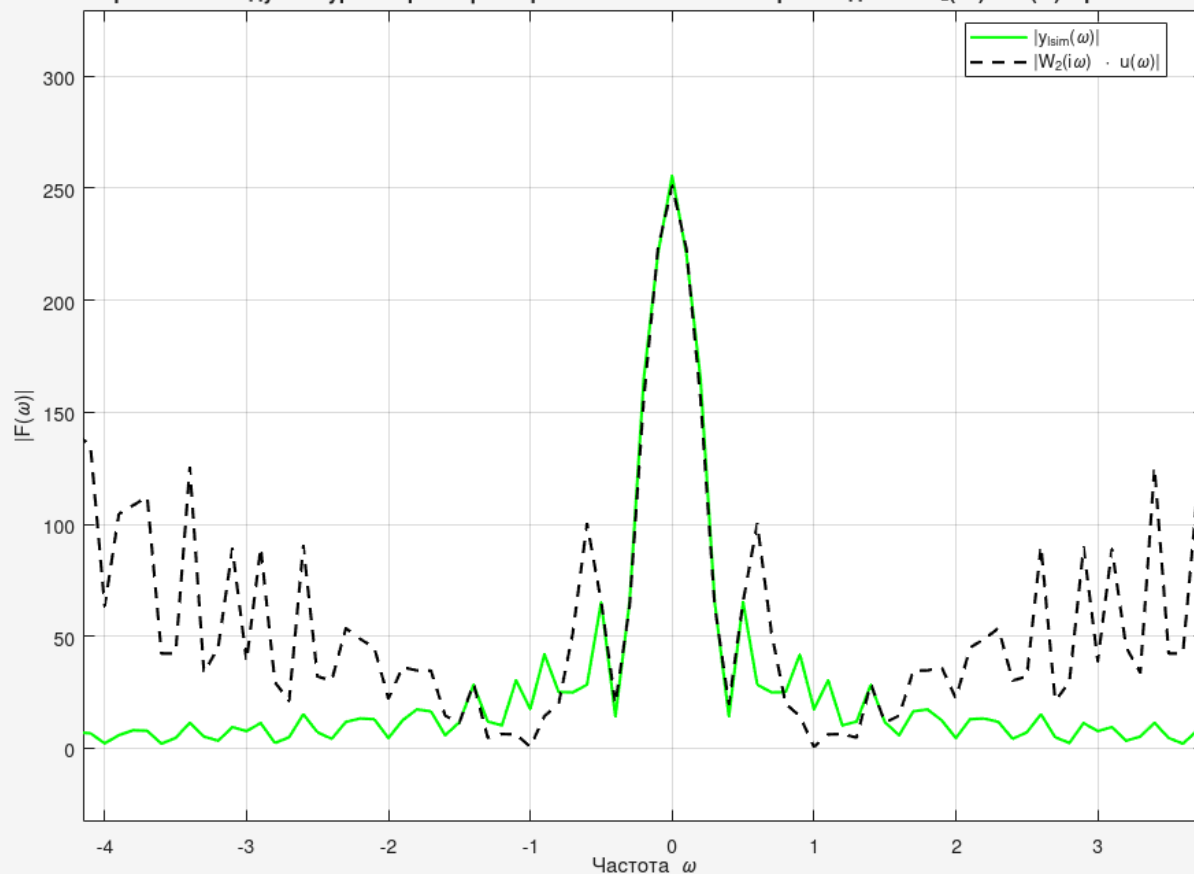


Когда частота помехи d совпадает с частотой подавления фильтра ω_0 (случай $d = \omega_0$, $d = 6$), фильтр работает наиболее эффективно. Если частота помехи d отличается от ω_0 , фильтр будет подавлять ее слабее, так как АЧХ на этой частоте не равна нулю.

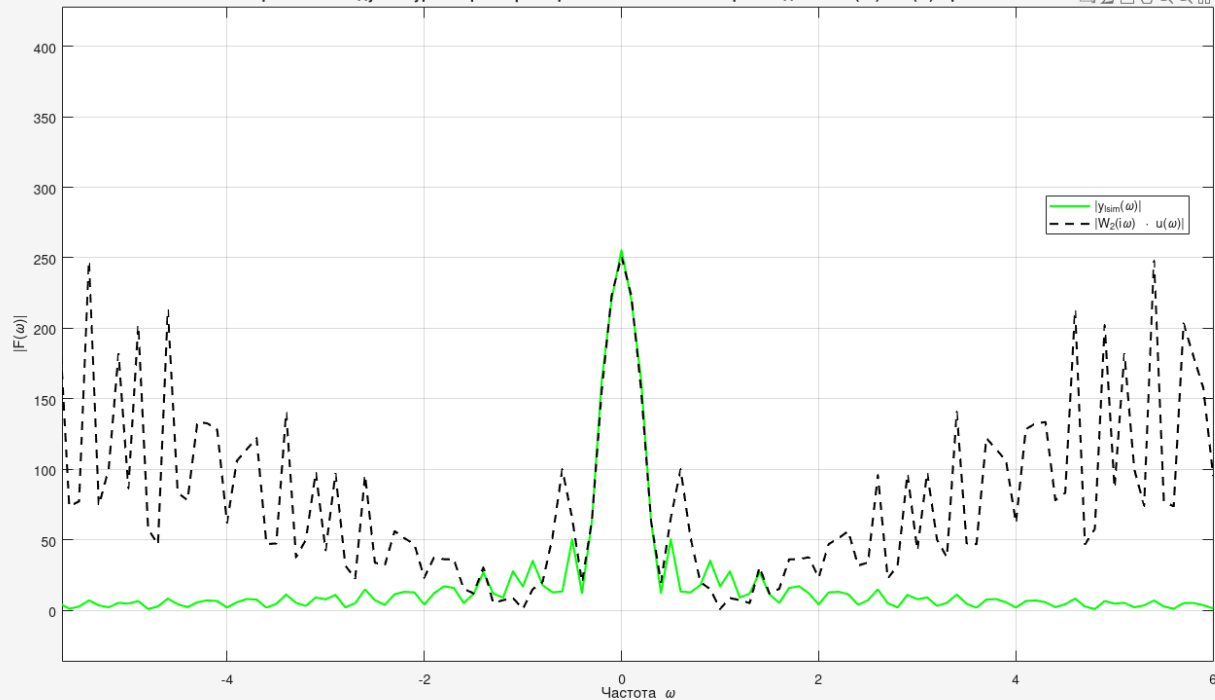
Проверка теоремы о свертке (сравнение $|y_{\text{lim}}(\omega)|$ и $|W_2(i\omega) \cdot u(\omega)|$)

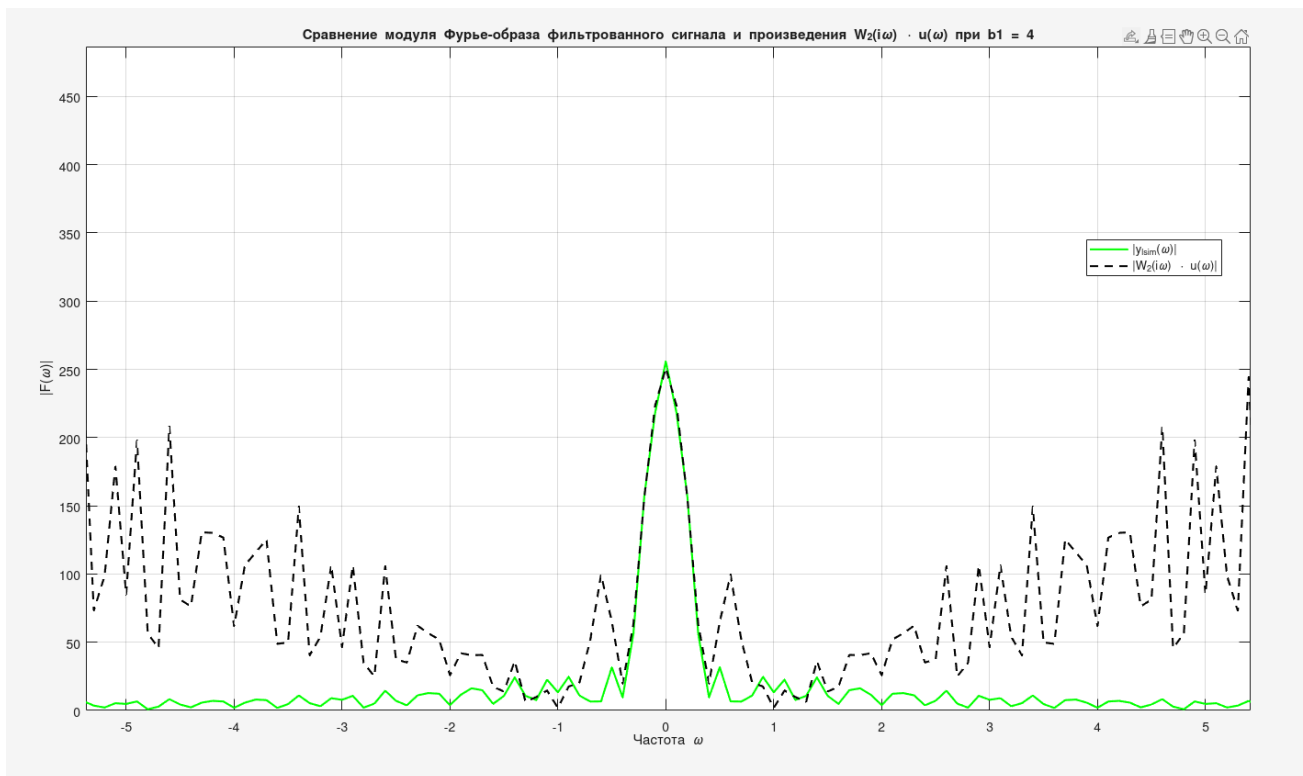


Сравнение модуля Фурье-образа фильтрованного сигнала и произведения $W_2(i\omega) \cdot u(\omega)$ при $b_1 = 1$



Сравнение модуля Фурье-образа фильтрованного сигнала и произведения $W_2(i\omega) \cdot u(\omega)$ при $b_1 = 2$





Графики показывают, что фильтрация с помощью функции `lsim` и фильтрация в частотной области (через FFT) дают практически идентичные результаты, т.е. теорема о свертке подтверждена: фильтрация во временной области эквивалентна умножению в частотной области. FFT может быть быстрее для длинных сигналов, но требует внимательного обращения с граничными условиями и масштабированием.

Вывод по п. 1.2

Эффективность режекторного фильтра напрямую зависит от правильного выбора параметров, особенно частоты подавления и коэффициента демпфирования. Для практического применения необходимо точно знать частоту помехи и выбирать параметры фильтра, исходя из компромисса между качеством подавления помехи и сохранением полезного сигнала.

Задание 2. Сглаживание биржевых данных

Код см. в приложении 2.1

Применим фильтр первого порядка для улучшения читаемости графиков

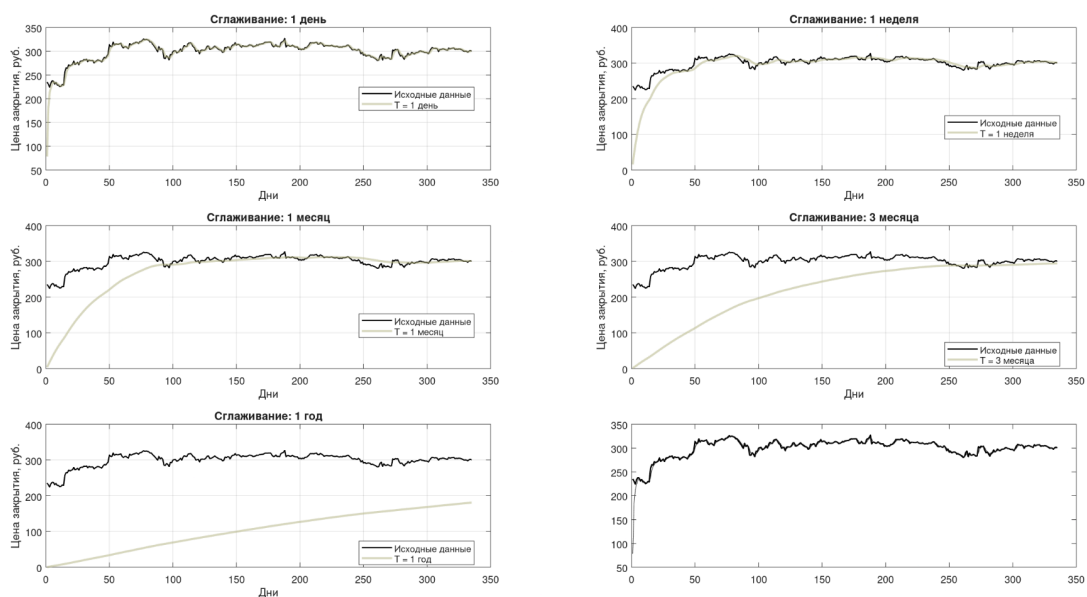
$$W(p) = \frac{1}{Tp + 1},$$

T - постоянная времени фильтра

Таблица SBER.csv выглядит таким образом (всего 336 записей)

	A	B	C	D	E	F	G	H	I
1	<TICKER>	<PER>	<DATE>	<TIME>	<OPEN>	<HIGH>	<LOW>	<CLOSE>	<VOL>
2	SBER	D	241202	0	237.09	238.7	234.32	235.17	45050170
3	SBER	D	241203	0	235.29	235.97	230.26	230.8	45313800
4	SBER	D	241204	0	231.05	233.79	224.3	224.55	72374990
5	SBER	D	241205	0	224.6	233.75	222.82	233.48	86902200
6	SBER	D	241206	0	233.63	238.25	231.52	237.84	81003410
7	SBER	D	241209	0	239.27	240.51	236.75	237.29	52314320
8	SBER	D	241210	0	238	238.01	230.65	230.82	58548350
9	SBER	D	241211	0	230.5	234.39	229.22	234.19	64817940

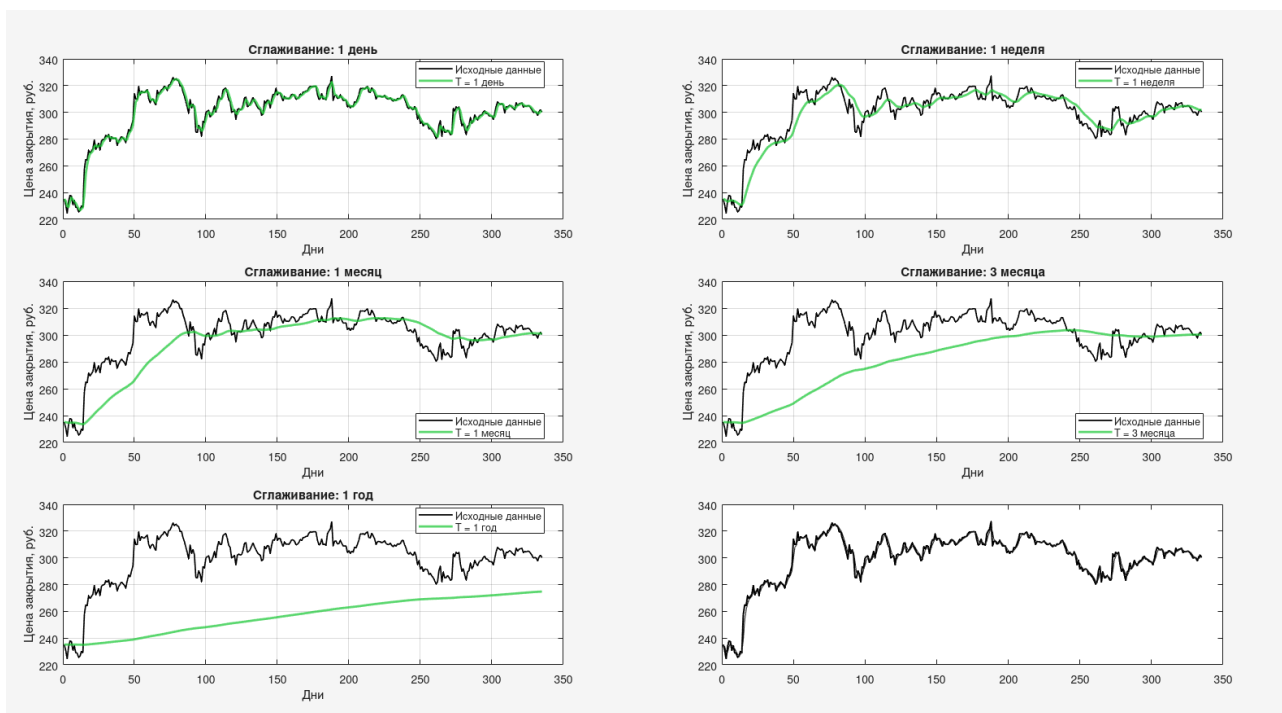
Основной анализ проводился по столбцу <Close> - цена закрытия торговой сессии.



(lsim по умолчанию использует нулевые начальные условия, из-за чего фильтрованный сигнал начинается с нуля, даже если исходные котировки находятся на уровне 200–300 рублей. Это визуально искажает начало графика)

Это можно исправить, если использовать метод *filter* с дискретной системой и начальными условиями, равными первому значению.

```
zi = filtic(num, den, closePrices(1), closePrices(1));  
filtered_signals(:, k) = filter(num, den, closePrices, zi);
```



Выводы по п. 2.0

На графиках видно, как увеличение постоянной времени T приводит к сильному сглаживанию, увеличению фазовой задержки (максимумы и минимумы смещаются вправо), подавлению краткосрочных всплесков, что полезно для анализа долгосрочных трендов.

Наименьшее значение $T=1$ день практически не изменяет сигнал — фильтр пропускает почти все колебания.

Приложения

```
clear; close all; clc;

%% Параметры сигнала
a = 1;           % Амплитуда импульса
t1 = -1;         % Начало импульса
t2 = 1.5;        % Конец импульса

%% Параметры шума и фильтра
b = 0.5;
c = 0;
d = 10;

T_const = 0.3; % Постоянная времени фильтра

%% Параметры дискретизации
T_total = 8; % Общая длительность сигнала (сек)
dt = 0.001;
t = -T_total/2 : dt : T_total/2;
N = length(t);

%% g(t)
g = zeros(size(t));
g(t >= t1 & t <= t2) = a;

%% u(t)
xi = 2 * rand(size(t)) - 1; % Равномерный шум U[-1, 1]
u = g + b * xi + c * sin(d * t);

%% фильтр W1(p) = 1 / (T_const * p + 1)
W1 = tf(1, [T_const, 1]);

%% Пропуск через фильтр во временной области
y_time = lsim(W1, u, t);

V = 1 / dt;
dv = 1 / T_total;
v = -V/2 : dv : V/2;
omega = 2 * pi * v;

%% FFT
U_fft = fftshift(fft(u));

W1_freq = arrayfun(@(w) 1 ./ (1j * w * T_const + 1), omega);

Y_freq = W1_freq .* U_fft;
y_freq = ifft(ifftshift(Y_freq));

%% === OUT ===

figure;
plot(t, g, 'k', 'LineWidth', 1.5); hold on;
plot(t, u, 'Color', [0, 0, 1, 0.1], 'LineWidth', 1);
plot(t, y_time, 'r', 'LineWidth', 1.2);
legend('g(t) – исходный', 'u(t) – зашумлённый', 'y(t) – фильтрованный (lsim)');
xlabel('Время t'); ylabel('Амплитуда');
title('Сигналы во временной области');
grid on;

% 2. Сравнение y_time и y_freq
figure;
plot(t, y_time, 'r', 'LineWidth', 1.2); hold on;
plot(t, real(y_freq), 'g--', 'LineWidth', 1);
legend('lsim', 'частотная фильтрация');
title('Сравнение методов фильтрации');
xlabel('Время t'); ylabel('Амплитуда');
```

```

grid on;

% 3. АЧХ фильтра
figure;
subplot(2,1,1);
semilogx(omega, abs(w1_freq));
xlabel('\omega (рад/с)'); ylabel('|W_1(i\omega)|');
title('АЧХ фильтра первого порядка');
grid on;

% 4. Спектры Фурье
G_fft = fftshift(fft(g));
Y_time_fft = fftshift(fft(y_time));

subplot(2,1,2);
plot(omega, abs(G_fft)/N, 'k', 'LineWidth', 1); hold on;
plot(omega, abs(U_fft)/N, 'b', 'Alpha', 0.6);
plot(omega, abs(Y_time_fft)/N, 'r', 'LineWidth', 1);
xlim([-50, 50]); % Убираем "хвосты" для наглядности
xlabel('\omega (рад/с)'); ylabel('|Спектр|');
legend('|ĝ(ω)|', '|û(ω)|', '|ŷ(ω)|');
title('Спектры сигналов');
grid on;

```

Приложение 1.1: исходный код программы (коэф. менялись в процессе экспериментов)

```

clc; clear; close all;

% g(t)
a = 1.0;
t1 = -1.0;
t2 = 1.5;

% Параметры синусоиды
c = 0.5;
d = 4.0;

T_total = 10;
dt = 0.01;
t = -T_total/2 : dt : T_total/2;

% Параметры фильтра
% W2(p) = (p^2 + a2) / (p^2 + b1*p + b2)
% где a2 = b2 = ω0^2, а b1 > 0 для устойчивости.
omega0 = d; % Частота, которую мы хотим подавить
a1 = 0;
a2 = omega0^2;
b2 = omega0^2;

b1_values = [0.5, 1.0, 2.0, 4.0]; % Разные значения демпфирования

% g(t)
g = zeros(size(t));
g(t >= t1 & t <= t2) = a;

% u(t) (без белого шума, только синусоида)
u = g + c * sin(d * t);

%% 2. Расчет и построение АЧХ фильтра для разных b1
figure('Name', 'АЧХ фильтра');
hold on;
grid on;
title('Амплитудно-частотная характеристика (АЧХ) режекторного фильтра');

```



```

xlabel('Частота \omega');
ylabel('|W_2(i\omega)|');

for i = 1:length(b1_values)
    b1 = b1_values(i);

    num = [1, a1, a2]; % p^2 + a1*p + a2
    den = [1, b1, b2]; % p^2 + b1*p + b2
    W2 = tf(num, den);

    % Расчет АЧХ
    w = logspace(-1, 1, 1000);
    [mag, ~] = bode(W2, w);
    mag = squeeze(mag);

    % Построение АЧХ
    plot(w, mag, 'LineWidth', 1.5, 'DisplayName', ['b1 = ', num2str(b1)]);
end

line([omega0, omega0], [0, 1], 'Color', 'r', 'LineStyle', '--', 'LineWidth', 2, ...
    'DisplayName', ['\omega_0 = ', num2str(omega0)]);
legend('Location', 'best');
xlim([0.1, 10]); % Ограничим ось X для наглядности
ylim([0, 1.2]);

%% 3. Исследование влияния параметров и построение сравнительных графиков

for idx_b1 = 1:length(b1_values)
    b1 = b1_values(idx_b1);

    num = [1, a1, a2];
    den = [1, b1, b2];
    W2 = tf(num, den);

    [y_lsim, ~] = lsim(W2, u, t);

    N = length(u);
    U_fft = fftshift(fft(u));
    V = 1/dt;
    dv = 1/T_total;
    v = -V/2 : dv : V/2;

    W2_iw = (1i*v).^2 + a2 ./ ((1i*v).^2 + b1*(1i*v) + b2);

    W2_iw(isnan(W2_iw)) = 0;
    W2_iw(isinf(W2_iw)) = 0;

    Y_fft = W2_iw .* U_fft;

    y_freq = ifft(ifftshift(Y_fft));
    y_freq = real(y_freq);

    % Построение сравнительных графиков сигналов во временной области
    figure('Name', ['Сигналы при b1 = ', num2str(b1)]);
    subplot(2,1,1);
    plot(t, g, 'b-', 'LineWidth', 1.5, 'DisplayName', 'Исходный сигнал g(t)');
    hold on;
    plot(t, u, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Зашумленный сигнал u(t)');
    plot(t, y_lsim, 'g-', 'LineWidth', 1.5, 'DisplayName', 'Фильтрованный сигнал (lsim)');
    plot(t, y_freq, 'm--', 'LineWidth', 1.5, 'DisplayName', 'Фильтрованный сигнал (FFT)');
    title(['Сравнение сигналов при b1 = ', num2str(b1)]);
    xlabel('Время t');
    ylabel('Амплитуда');
    grid on;
    legend('Location', 'best');

```

```

xlim([-3, 3]); % Ограничиваем для лучшей видимости импульса

% Построение сравнительных графиков модулей Фурье-образов
subplot(2,1,2);
% Вычисляем Фурье-образы
G_fft = fftshift(fft(g));
U_fft_abs = abs(U_fft);
Y_lsim_fft = fftshift(fft(y_lsim));
Y_lsim_fft_abs = abs(Y_lsim_fft);
Y_freq_fft_abs = abs(Y_fft); % Уже в правильном порядке после ifftshift

freq_lim = 6; % Смотрим только до 6 рад/с
idx_lim = find(abs(v) <= freq_lim, 1, 'last');
v_plot = v(1:idx_lim);
G_fft_abs_plot = abs(G_fft(1:idx_lim));
U_fft_abs_plot = U_fft_abs(1:idx_lim);
Y_lsim_fft_abs_plot = Y_lsim_fft_abs(1:idx_lim);
Y_freq_fft_abs_plot = Y_freq_fft_abs(1:idx_lim);
W2_iw_plot = abs(W2_iw(1:idx_lim));

plot(v_plot, G_fft_abs_plot, 'b-', 'LineWidth', 1.5, 'DisplayName', '|g(\omega)|');
hold on;
plot(v_plot, U_fft_abs_plot, 'r--', 'LineWidth', 1.5, 'DisplayName', '|u(\omega)|');
plot(v_plot, Y_lsim_fft_abs_plot, 'g-', 'LineWidth', 1.5, 'DisplayName',
|y_{lsim}(\omega)|');
plot(v_plot, Y_freq_fft_abs_plot, 'm--', 'LineWidth', 1.5, 'DisplayName',
|y_{freq}(\omega)|');
title(['Модули Фурье-образов при b1 = ', num2str(b1)]);
xlabel('Частота \omega');
ylabel('|F(\omega)|');
grid on;
legend('Location', 'best');

figure('Name', ['Сравнение Фурье-образов при b1 = ', num2str(b1)]);
plot(v_plot, Y_lsim_fft_abs_plot, 'g-', 'LineWidth', 1.5, 'DisplayName',
|y_{lsim}(\omega)|');
hold on;
plot(v_plot, W2_iw_plot .* U_fft_abs_plot, 'k--', 'LineWidth', 1.5, 'DisplayName',
|W_2(i\omega) \cdot u(\omega)|');
title(['Сравнение модуля Фурье-образа фильтрованного сигнала и произведения W_2(i\omega)
\cdot u(\omega) при b1 = ', num2str(b1)]);
xlabel('Частота \omega');
ylabel('|F(\omega)|');
grid on;
legend('Location', 'best');
end

```

%% 4. Исследование влияния параметра d (частоты помехи)

```

b1_fixed = 1.0; % Фиксируем значение b1
d_values = [2.0, 4.0, 6.0];

figure('Name', 'Влияние частоты помехи d');
for idx_d = 1:length(d_values)
    d = d_values(idx_d);

    u_new = g + c * sin(d * t);

    omega0 = d;
    a2 = omega0^2;
    b2 = omega0^2;
    num = [1, a1, a2];
    den = [1, b1_fixed, b2];
    W2 = tf(num, den);

```

```

[y_lsim, ~] = lsim(W2, u_new, t);

subplot(length(d_values), 1, idx_d);
plot(t, g, 'b-', 'LineWidth', 1.5, 'DisplayName', 'g(t)');
hold on;
plot(t, u_new, 'r--', 'LineWidth', 1.5, 'DisplayName', 'u(t)');
plot(t, y_lsim, 'g-', 'LineWidth', 1.5, 'DisplayName', 'y(t)');
title(['Сигналы при d = ', num2str(d)]);
xlabel('Время t');
ylabel('Амплитуда');
grid on;
legend('Location', 'best');
xlim([-3, 3]);
end

```

Приложение 1.2: исходный код программы

```

clear; clc; close all;

rawText = fileread('SBER.csv');
lines = strsplit(rawText, '\n');
lines = lines(~cellfun(@isempty, lines));

dataLines = lines(2:end);

closePrices = zeros(length(dataLines), 1);

for i = 1:length(dataLines)
    line = dataLines{i};
    line = strrep(line, '\;', ';');
    line = strrep(line, '\\', '');
    parts = strsplit(line, ';');
    if length(parts) >= 8
        valStr = parts{8};
        val = str2double(valStr);
        if ~isnan(val)
            closePrices(i) = val;
        else
            closePrices(i) = NaN;
        end
    else
        closePrices(i) = NaN;
    end
end

closePrices = closePrices(~isnan(closePrices));
N = length(closePrices);
t_days = (1:N)';

fprintf('Загружено %d значений цены закрытия.\n', N);

T_values = [1, 7, 30, 90, 365];
T_names = {'1 день', '1 неделя', '1 месяц', '3 месяца', '1 год'};
dt = 1;

%% Фильтрация с помощью filter и начальных условий
filtered_signals = zeros(N, length(T_values));

for k = 1:length(T_values)
    T = T_values(k);
    fprintf('Применение фильтра с T = %s (%d дней)...\n', T_names{k}, T);

    sys_cont = tf(1, [T, 1]);

    sys_disc = c2d(sys_cont, dt, 'tustin');

    [num, den] = tfdata(sys_disc, 'v');

```

```

        zi = filtic(num, den, closePrices(1), closePrices(1));
        filtered_signals(:, k) = filter(num, den, closePrices, zi);
    end

    fprintf('Фильтрация завершена.\n');

    %% Построение графиков
    figure('Name', 'Задание 2: Сглаживание котировок Сбербанка', ...
        'Position', [100, 50, 1300, 900]);

    colors = lines(5);

    for k = 1:5
        subplot(3, 2, k);
        plot(t_days, closePrices, 'k-', 'LineWidth', 1.2, 'DisplayName', 'Исходные данные');
        hold on;
        plot(t_days, filtered_signals(:, k), 'Color', colors(k,:), 'LineWidth', 2, ...
            'DisplayName', ['T = ', T_names{k}]);
        hold off;
        grid on;
        xlabel('Дни');
        ylabel('Цена закрытия, руб.');
        title(['Сглаживание: ', T_names{k}]);
        legend('Location', 'best');
    end

    subplot(3, 2, 6);
    plot(t_days, closePrices, 'k-', 'LineWidth', 1.4, 'DisplayName', 'Исходные');
    hold on;
    for k = 1:5
        plot(t_days, filtered_signals(:, k), 'Color', colors(k,:), 'LineWidth', 1.5, ...
            'DisplayName', ['T = ', T_names{k}]);
    end
    hold off;
    grid on;
    xlabel('Дни');
    ylabel('Цена закрытия, руб.');
    title('Все варианты сглаживания');
    legend('Location', 'best');

    sgtitle('Сглаживание биржевых данных фильтром первого порядка (Задание 2)', 'FontSize', 14);

```

Приложение 2.0: исходный код программы (с исправлением через filter)